A Secure Connected Measurement Node for Greenhouse Digital Twins: from an Autonomous Harvester to an Authenticated Robot–Cloud Telemetry Chain

Franck Armand Josias Einstein
Projet de Fin d'Année (PFA)
Smart Agriculture — Digital Twin
Email: josiasarmand40@gmail.com

***Abstract*—This report documents the design, implementation and experimental evaluation of a *connected measurement node* for precision agriculture: a mobile robot that receives cryptographically signed work orders from a Cloud service, drives itself to painted reference marks in a greenhouse, records the plant environment, and returns the readings sealed against both disclosure and forgery. The work spans three phases on one KUKA YouBot platform. Phase A (Webots) and Phase B (ROS 2 Jazzy / Gazebo Harmonic) build a full autonomous mobile-manipulation stack — mecanum kinematics, log-odds occupancy mapping, A*, pure pursuit, correlative scan-matching SLAM, a five-DOF arm, and a measurement layer scoring pose and map against simulator ground truth. Phase C, the delivered system and the body of this report, answers a different question: not *how does a robot explore an unknown space*, but *how does a fleet operator obtain trustworthy data from a known one*. Its greenhouse is a generated twin of the Phase B scene carrying 48 painted station marks derived from a single coordinate catalogue, so the world the robot drives and the map it reasons from cannot drift apart. Navigation is deterministic and closed-loop — a three-leg geometric route, a lidar brake that distinguishes catalogued structure from genuine obstruction, and a downward camera that closes the last centimetres on the painted mark. The camera resolved its mark at 48 of 48 stations in the reference campaign, holding every visit inside the 0.04 m tolerance that the 0.10 m spacing of adjacent stations imposes. Each reading is encoded as a scannable QR code, photographed, and sealed with ECIES over SECP256R1 and an ECDSA-P256 signature; the Cloud refuses, counts and displays anything that fails to verify. Ordering the 48 stations by free-space band rather than by catalogue index cuts the driven distance from 312 m to 40 m and the measured campaign time from 1 505 s to 601 s, and the accompanying power model shows why that is the smaller half of the energy bill. A 384-check pre-flight suite, each check encoding a failure the project actually suffered, runs in fifteen seconds with no broker, no ROS and no network. Negative and modelled results are reported with the same emphasis as positive ones, and the capabilities built in Phases A–B but not exercised by the delivered mission are set out explicitly as the platform's extension path.**

***Index Terms*—Precision agriculture, agricultural IoT, digital twin, connected node, elliptic-curve cryptography, ECIES, MQTT, ROS 2, Gazebo, KUKA YouBot, visual servoing, fiducial alignment, route optimisation, energy budgeting, regression testing, agricultural robotics.**

I. Introduction

A. Context and motivation

Strawberry production is among the most labour-intensive branches of horticulture. Harvesting alone accounts for a large share of the operating cost of a table-top greenhouse, it must be repeated every two to three days through the season, and it is increasingly constrained by the availability of seasonal labour. The task is also unusually hard to automate: the fruit is soft, it grows in clusters where ripe and unripe berries touch, it is partly hidden by leaves, and the picking window is short. These properties have made strawberry harvesting a recurring benchmark for agricultural robotics rather than a solved problem [1], [2].

A greenhouse, on the other hand, is a far friendlier environment for a mobile robot than an open field. It is flat, indoor, geometrically regular, free of weather, and its layout — parallel raised gutters separated by service aisles — is known in advance and does not change between seasons. That combination is what makes a *digital twin* attractive: because the physical structure is stable and measurable, a simulated greenhouse can be made dimensionally faithful, and a control stack developed against it has a realistic chance of transferring to the real installation.

B. Objective of this work

The objective of this Projet de Fin d'Année is to build, from nothing, a complete autonomous stack that lets a KUKA YouBot omnidirectional mobile manipulator:

1) build a metric map of an unknown greenhouse from a 2D lidar;
2) localise itself in that map while it moves;
3) plan and follow collision-free paths along the service aisles;
4) detect ripe strawberries with an on-board colour camera and place them on the map;
5) drive its five-DOF arm to pick them; and
6) repeat the cycle autonomously, delivering the fruit to a depot.

A secondary objective, imposed by the intended reuse of the platform in healthcare logistics and indoor transport, is that the navigation, mapping and safety layers must be *domain-independent*: nothing in them may assume strawberries, gutters, or this particular greenhouse.

C. Two phases, and why

The work is presented chronologically because it was carried out chronologically, and because the second phase is only intelligible as a consequence of the first.

Phase A (Webots). A complete prototype was first written for Webots R2025a, in plain Python, with no middleware. This was deliberate: Webots supplies the robot model, the physics and the sensors out of the box, so the entire effort could go into the algorithms themselves — mecanum kinematics, odometry, occupancy mapping, A*, pure pursuit, colour vision, arm sequencing, and a behaviour-tree mission supervisor. Fifteen controllers were written, ten of them single-purpose test controllers used to validate one subsystem at a time. Phase A produced 3 844 lines of Python and, more importantly, an algorithm layer that had been exercised against a working simulator.

Phase B (ROS 2 / Gazebo). The validated algorithms were then lifted, essentially unmodified, into a framework-independent library (`youbot_control/lib/`) and wrapped in ROS 2 Jazzy nodes, with the scene rebuilt as a Gazebo Harmonic digital twin measured against the real greenhouse. Phase B is therefore not a rewrite but a *re-architecture*: the same A*, the same pure pursuit, the same log-odds grid, now distributed across processes, with the problems that distribution creates — and with a measurement layer added so that those problems become visible instead of being guessed at.

D. What this report claims, and what it does not

This report is written to be checked. Every quantitative statement in it is traceable to a specific line of a specific simulation log, and the procedure to regenerate those logs is given in Appendix A. Where a subsystem does not work, that is stated with the same emphasis as where one does. In particular:

- The mapping subsystem works well. The reconstructed interior is accurate to -4 **cm** / $+10$ **cm** against a 9.90×4.90 m reference, with **100%** of the free floor observed and **0.0%** of it wrongly marked as obstacle (Sec. XVII).
- Odometry calibration works. A one-off UMBmark-style calibration [3] holds position error at **0.20 m** over a full mission, where the uncalibrated odometry diverges past **10 m**.
- *The homemade SLAM module does not work.* Incremental correlative scan matching without loop closure was implemented, instrumented, and measured to be *worse* than the odometry prior it corrects. Sec. V-D explains why this is a property of the algorithm class and not a coding defect, and the production configuration disables it.
- *The fruit detector is not accurate enough.* Colour thresholding reaches **77%** recall but only about one correct estimate in three, at **0.22 m** localisation error — roughly a quarter of the plant spacing, which means it identifies the right plant but not the right berry.

Reporting the negative results in full is not modesty. The instrumented comparison between a homemade SLAM front-end and a calibrated dead-reckoning baseline is one of the more useful outcomes of the project, and it only exists because the measurement layer was built before the conclusions were drawn.

E. Contributions

1) A complete, documented, reproducible two-phase implementation of an autonomous greenhouse harvester, from world modelling to mission supervision, with every algorithm re-implemented rather than imported from a navigation stack.
2) A framework-independent algorithm library validated in one simulator (Webots) and reused unchanged in another (Gazebo/ROS 2), demonstrating a practical portability pattern.
3) A three-axis measurement layer — pose accuracy, map accuracy, and time/throughput — that scores the running system against simulator ground truth continuously, and which caught two regressions that no test and no visual inspection had caught.
4) A pre-flight regression suite of 191 pure-Python checks, each one encoding a failure the project actually suffered, that runs in about one second before every simulation.
5) An explicit SWOT analysis of every algorithm and architectural decision (Sec. XVI), including the ones that were abandoned.

F. How to read the flowcharts

Every control-flow structure in this report is given as a flowchart using one fixed visual language, defined once in Fig. 1 and never redefined afterwards. Two conventions matter:

Shapes follow classic flowchart practice, with two additions specific to a robotics stack: a distinct shape for a *message read or written on a ROS topic*, and a distinct shape for a *call into the framework-independent algorithm library*. Being able to see, at a glance, where a diagram touches the middleware and where it touches the algorithms is the whole point of the split described in Sec. V-B.

Phase bands partition every diagram into three coloured regions: **INITIALISATION** (executed once, before any data arrives), **MAIN LOOP** (the periodic callback, executed at a fixed rate for the whole run), and **TERMINATION / DESTRUCTION** (shutdown, resource release, final reporting). The banding is not decoration: a recurring source of defects in this project was code placed in the wrong band — state armed once that should have been re-armed per cycle, and cleanup that never ran because it was inside the loop it was meant to end.

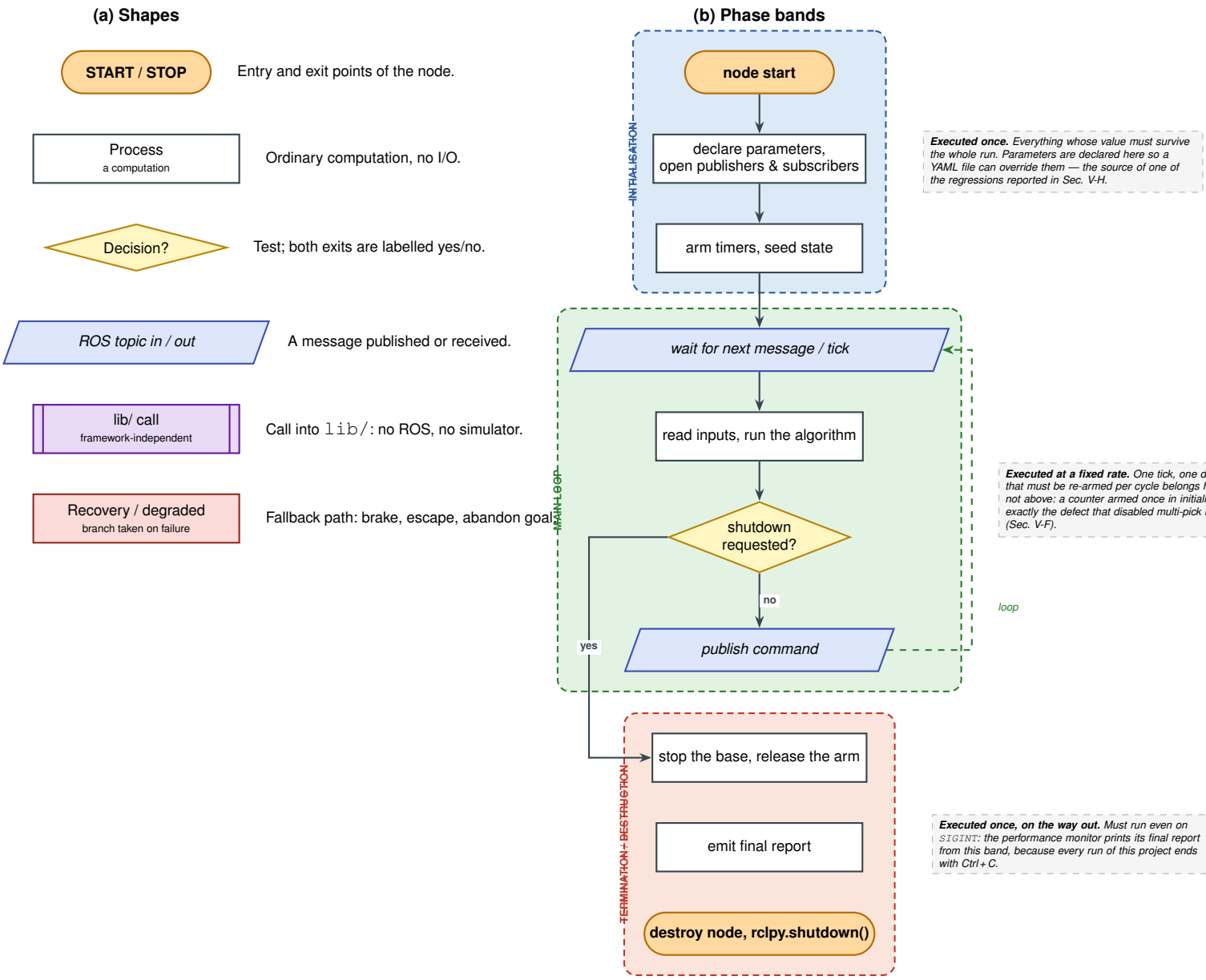


Figure 1: The flowchart language used throughout this report, defined once. (a) The six node shapes: two are specific to a robotics stack — a *ROS topic* shape, so the boundary with the middleware is visible, and a *library call* shape, so the boundary with the framework-independent algorithm layer is visible. (b) The three phase bands into which every later flowchart is partitioned. The banding is functional rather than decorative: two of the defects analysed in this report are cases of code placed in the wrong band.

G. Report structure

Sec. II reviews the literature each algorithm is drawn from. Sec. III specifies the toolchain, the exact software versions, and the installation procedure. Sections IV–IV-M present Phase A (Webots) in full, subsystem by subsystem. Sections V–V-H present Phase B (ROS 2 / Gazebo) in the same order, so the two phases can be read side by side. Sec. XVI gives the SWOT analysis of every method. Sec. XVII reports the measured results, Sec. XX discusses the limitations and what they imply for a physical deployment, and Sec. XXI concludes. Appendix A gives the commands to reproduce every experiment; Appendix B tabulates every tuned parameter with its justification; Appendix C gives the full topic contract; Appendix D collects the failure modes encountered and their diagnosis.

II. Related Work

[Section in preparation.]

Table I: Development and experiment platform

Item	Value
Machine	HP EliteBook laptop
Operating system	Ubuntu 24.04 LTS (Noble Numbat)
Display server	Wayland, with Qt forced to XCB
Python	3.12 (system interpreter)
Project storage	external volume, mounted under `/mnt`

III. Development Environment and Toolchain

This section specifies the software environment precisely enough that every result in this report can be regenerated on a clean machine. It is placed before the technical chapters deliberately: several of the defects analysed later are consequences of version-specific behaviour (a deprecated Gazebo service, a build mode that resolves entry points through a symbolic link), and those are not intelligible without knowing exactly what was running.

A. Hardware and operating system

All development and all reported experiments were carried out on a single workstation, Table I. No GPU-accelerated inference is used anywhere in the stack; the only GPU consumer is the simulator's rendering pipeline, which also serves the simulated lidar (Sec. V-A explains why that matters).

The display server is worth recording. Gazebo's GUI detects Wayland and falls back to the XCB Qt platform plugin, printing `Detected Wayland. Setting Qt to use the xcb plugin` at every start. This is benign in itself, but it is one reason the GUI camera-tracking behaviour differs from the documented default, which motivated the camera watchdog described in Sec. V-A.

B. Software versions

Table II lists every component whose version affects the results. Two entries are load-bearing:

- **Gazebo Harmonic (gz-sim 8.11.0)** deprecated the `/gui/follow` service in favour of the `/gui/track` topic. Code written against the older API fails silently — the camera simply never locks onto the robot — which cost one full debugging cycle (Appendix D).
- **`colcon build --symlink-install`** resolves console-script entry points through an egg-link back into `src/`. When that metadata goes missing, every node dies at import with `PackageNotFoundError` — fifteen identical tracebacks that name the symptom and not the cause. The launch wrapper tests for this explicitly before starting anything.

C. Installation

Listing 1 gives the complete installation from a clean Ubuntu 24.04. The ROS 2 and Gazebo repositories are added first; the `ros-jazzy-ros-gz` metapackage pulls in both the bridge and the simulator integration.

Table II: Software components and versions

Component	Version	Role
Webots	R2025a	Phase A simulator
ROS 2	Jazzy Jalisco	Phase B middleware
Gazebo Sim	Harmonic 8.11.0	Phase B simulator
`ros_gz_sim`	Jazzy	spawn, world launch
`ros_gz_bridge`	Jazzy	topic bridging
`webots_ros2_driver`	Jazzy	Phase A→ROS
RViz 2	Jazzy	visualisation
NumPy	system package	grids, scan matching
colcon	system package	build tool
Physics engine	DART (default)	1 ms step

```
# --- 1. ROS 2 Jazzy ----------------------------------
sudo apt update && sudo apt install -y \
     software-properties-common curl
sudo add-apt-repository universe
sudo curl -sSL -o /usr/share/keyrings/ros-archive-keyring.gpg \
  https://raw.githubusercontent.com/ros/rosdistro/master/ros.key
echo "deb [signed-by=/usr/share/keyrings/ros-archive-keyring.gpg] \
http://packages.ros.org/ros2/ubuntu noble main" \
     | sudo tee /etc/apt/sources.list.d/ros2.list
sudo apt update
sudo apt install -y ros-jazzy-desktop

# --- 2. Gazebo Harmonic + the ROS 2 bridge -----------
sudo apt install -y ros-jazzy-ros-gz \
                    ros-jazzy-ros-gz-sim \
                    ros-jazzy-ros-gz-bridge

# --- 3. Webots R2025a (Phase A) ----------------------
sudo apt install -y ros-jazzy-webots-ros2
# plus the Webots R2025a .deb from cyberbotics.com

# --- 4. Build tooling and Python dependencies --------
sudo apt install -y python3-colcon-common-extensions \
                    python3-numpy python3-opencv

# --- 5. The report toolchain (optional) --------------
sudo apt install -y texlive-latex-extra texlive-publishers \
                    texlive-science
```

Listing 1: Full installation on a clean Ubuntu 24.04.

D. Workspace layout and build

The ROS 2 workspace contains five packages, Table III. The split is not cosmetic: `youbot_control` holds the algorithms and the control nodes and depends on *no* simulator, while `youbot_gazebo` and `youbot_webots` are thin, simulator-specific shells. This is what allowed Phase A to be reused in Phase B rather than rewritten (Sec. V-B).

Note the direction of the arrows: no package containing an algorithm depends on a simulator, and no simulator package is depended upon by another. `youbot_slam` depends on `youbot_control` only to reuse the occupancy-grid and scan-matching libraries.

Table III: ROS 2 workspace packages and their dependencies

Package	Depends on
youbot_control	rclpy, geometry_msgs, sensor_msgs, nav_msgs, tf2_ros, visualization_msgs, numpy
youbot_slam	rclpy, nav_msgs, sensor_msgs, geometry_msgs, tf2_ros, youbot_control
youbot_gazebo	youbot_control, youbot_bringup, ros_gz_sim, ros_gz_bridge, robot_state_publisher, rviz2
youbot_bringup	youbot_control, launch, launch_ros, rviz2, robot_state_publisher
youbot_webots	webots_ros2_driver, youbot_control, rclpy

```
git clone https://github.com/\
franckarmandjosiasEinstein-bit/ROS2_stage.git
cd ROS2_stage
source /opt/ros/jazzy/setup.bash
colcon build --symlink-install
source install/setup.bash
```

Listing 2: Cloning and building.

`--symlink-install` is used so that editing a Python node takes effect without a rebuild, which matters when one simulation run costs half an hour of wall time. Its price is the entry-point fragility described above; if that appears, a plain `colcon build` writes real distribution metadata and removes the dependency on `src/` entirely.

E. Running a simulation

Every experiment is launched through one wrapper script, `src/youbot_gazebo/scripts/run.sh`, rather than through `ros2 launch` directly. The wrapper exists because of three concrete failures, each of which cost at least one run:

1) **Gazebo does not die on Ctrl + C.** A surviving `gz sim` keeps publishing an old `/clock`; the next run then has two clocks, RViz thrashes, and the GUI can attach to a ghost world with no robot in it. The wrapper installs a cleanup trap and afterwards *verifies* that nothing survived, printing the command line of any process it could not match rather than leaving it to be inferred from the next run misbehaving.
2) **Gazebo rewrites its GUI configuration on every exit** with wherever the camera was left, and reads it back in preference to the world's `<gui>` pose. A run started after the camera was parked on empty space opens blind. The wrapper deletes `~/.gz/sim/8/gui.config` before starting.
3) **Silent build breakage**, via the entry-point check described above.

The wrapper then runs the pre-flight regression suite (Sec. V-H) and refuses to start if any check fails: about one second of pure Python weighed against thirty minutes of simulation.

```
# Level 2 -- ground-truth pose, no SLAM (baseline)
./src/youbot_gazebo/scripts/run.sh

# Level 3 -- homemade SLAM in the loop
./src/youbot_gazebo/scripts/run.sh slam

# Level 3 -- calibrated dead reckoning, SLAM
    ↪ disabled
#            (the production configuration, Sec. V
    ↪ -E)
./src/youbot_gazebo/scripts/run.sh slam \
      slam:=false pose_topic:=odom_calibrated

# Headless, for batch runs
./src/youbot_gazebo/scripts/run.sh slam gui:=false
```

Listing 3: The supported launch configurations.

A correct start prints three markers, in this order. Their absence is the fastest way to detect that a stale build is being run — a mistake made twice during this project, both times producing a log that looked plausible and was measuring the wrong binary:

```
[run.sh] pre-flight: all 191 checks passed.
[mapping_node] mapping_node up: /scan + /odom -> /
    ↪ map
                (inflated, for planning) + /map_raw
    ↪  (evidence)
[perf_monitor] perf_monitor up: timing, throughput
    ↪  and
                where the time goes.
```

F. Running Phase A (Webots)

Phase A is self-contained and does not require ROS. Webots is opened on one of the two worlds and the `controller` field of the YouBot node selects which of the fifteen controllers runs (Sec. IV). The `youbot_webots` package exists only to expose the same robot to ROS 2 through `webots_ros2_driver`; it was used to confirm that the algorithm layer behaved identically under both simulators before the Gazebo twin was built.

```
# Standalone Webots (no ROS): open the world, then
    ↪  choose
# the controller in the scene tree.
webots worlds/smart_agriculture.wbt
webots worlds/greenhouse.wbt

# The same robot exposed to ROS 2 (bridge
    ↪ validation)
ros2 launch youbot_webots webots.launch.py
```

Listing 4: Launching Phase A.

G. Version control and reproducibility

The two phases live in two repositories: the Webots prototype in `Projet`, and the ROS 2 stack in `ROS2_stage`. Every result quoted in Sec. XVII names the commit it was produced from, because two of the analyses in this report compare runs that differ by a single parameter file. Appendix A lists those commits alongside the exact command and the expected output.

IV. Phase A — Webots Prototype: Overview

A. Why a simulator-native prototype first

The first decision of the project was not which algorithm to use but *where to make the first mistake*. Two routes were available: start directly in ROS 2, where the target architecture lives, or start in Webots [4], where the robot model, the physics and the sensors are supplied and a controller is a single Python file with direct access to every device.

Webots was chosen, for three reasons that all proved correct in hindsight:

1) **The KUKA YouBot is a first-class Webots model.** Its mecanum base, its five-DOF arm and its two-finger gripper are modelled with the manufacturer's dimensions. No URDF had to be authored, no inertias guessed, no controller plugin written before the first wheel turned.
2) **One process, one debugger, no middleware.** A defect in a ROS 2 stack can be in the algorithm, in a QoS setting, in a remapping, in a transform, or in a launch file. In a Webots controller it can only be in the algorithm. Every hour spent debugging in Phase A was spent on robotics rather than on plumbing.
3) **Middleware is a re-architecture, not a rewrite.** If the algorithms are written without importing the simulator, moving them later is mechanical. Sec. V-B shows that this held: A*, pure pursuit, the log-odds grid, the mecanum kinematics and the scan matcher were carried into Phase B essentially unmodified.

The cost of the choice is that Phase A proves nothing about distributed execution, message timing, or transform trees — exactly the class of problem that dominated Phase B (Sec. V).

B. The Phase A mission

It is important to state plainly that Phase A and Phase B do not solve the same task. Phase A implements a *crate-collection* mission in a procedurally generated agricultural scene: the robot explores an arena containing six planting basins (`bassin_1`...`bassin_6`), discovers red crates with its camera, plans a route to each, picks it up with the arm, stows it in one of two body-frame slots on its rear plate, and delivers a full load to a depot before returning to explore again.

Phase B replaces the payload with strawberries growing on raised gutters. The mission logic changes; the navigation, mapping, planning and safety layers do not. That is precisely the domain-independence requirement stated in Sec. IV, and Phase A is where it was earned: nothing in `navigation.py`, `planning.py` or `mapping.py` mentions a crate.

C. Incremental validation: fifteen controllers

The distinguishing feature of Phase A is methodological rather than algorithmic. Instead of writing one controller and debugging it as a whole, *one single-purpose test controller was written per subsystem*, in dependency order, and each had to work in isolation before the next was started. Table IV lists all fifteen with their size.

Table IV: Phase A controllers, in the order they were written

Controller	LoC	Question it answers
Bring-up		
`device_scan`	61	What devices does this robot expose, and under what names?
`wheel_test`	99	Does each wheel turn, in the direction and at the rate commanded?
`base_test`	131	Do the mecanum equations produce forward, strafe and yaw correctly?
`odom_test`	115	Does wheel-encoder odometry track the ground-truth pose?
Perception		
`lidar_test`	149	Does the lidar produce a usable occupancy grid of a known scene?
`vision_test`	110	Are red crates segmented, and does the ground projection land on them?
`localize_test`	160	Does scan matching correct a deliberately drifted odometry?
Motion		
`planner_test`	115	Does A* return a collision-free path on the inflated grid?
`nav_test`	108	Does pure pursuit follow that path without cutting corners?
Manipulation		
`arm_test`	110	Does each arm joint reach its commanded angle?
`grasp_test`	170	Does the full pick sequence close on a crate and lift it?
Integration		
`world_manager`	185	Supervisor: generate the scene, score the mission, provide ground truth.
`my_First_ctrl`	603	The behaviour-tree mission that uses every module above.

Ten of the fifteen exist only to answer one question. This is deliberate. A test controller that fails tells you which subsystem is broken; an integrated controller that fails tells you only that something is. The discipline is also what made the port to Phase B tractable, because each module arrived at the port already having a known-good reference behaviour.

D. Module inventory

The mission controller itself is thin: it wires together eight modules, Table V, none of which imports Webots except through a small device-handle interface. The line counts are worth noting for what they say about where the difficulty lies: the behaviour tree and the localisation estimator are together larger than the planner, the mapper and the navigator combined.

E. Reading order for this chapter

The subsections that follow take the subsystems in the order they were built, which is also the order of Table IV: the world and its supervisor (Sec. IV-F), locomotion and odometry (Sec. IV-G), perception (Sec. IV-H), mapping and localisation (Sec. IV-I), planning and path following (Sec. IV-J), manipulation (Sec. IV-K), and finally the behaviour-tree mission that binds them (Sec. IV-L). Sec. IV-M reports what Phase A achieved and, more usefully, which of its design decisions had to be undone in Phase B.

Table V: Phase A algorithm modules

Module	LoC	Contents
behavior_trees	514	Blackboard, mission steps, and the py_trees behaviours
mapping	260	Log-odds occupancy grid, Bresenham ray casting, inflation
localization	252	Wheel odometry, likelihood-field scan matcher, fused estimator
navigation	236	Mecanum kinematics and pure pursuit
vision	169	Red-blob segmentation and ground back-projection
planning	139	A*, octile heuristic, path smoothing
exploration	96	Frontier extraction and waypoint ordering
arm	62	Joint poses, gripper open/close
Total (modules)	**1 728**	
Total (Phase A)	**3 844**	including all fifteen controllers

Table VI: Procedural scene parameters (world_manager.py)

Parameter	Meaning	Value
ARENA_HALF	arena half-extent	5.0 m
ROW_YS	basin row centre lines	$\pm 1.0, \pm 3.0$
POT_XS	basin positions per row	7 at 1.2 m
POT_SKIP_PROB	probability a basin is omitted	0.25
AISLE_YS	free corridors	$-2.0, 0.0, 2.0$
N_CRATES	mission objectives	2–5
N_WORKERS	moving obstacles	2
WORKER_SPEED	obstacle speed	0.30 m/s
KEEP_CLEAR	protected zones	start, depot
CLEAR_RADIUS	protected radius	1.0 m

F. World Modelling and the Procedural Greenhouse

1) Two worlds, two purposes: Phase A uses two Webots worlds. smart_agriculture.wbt (258 lines) is the working scene: a 10 m square arena with a rectangular floor, four rows of planting basins, a robot start corner and a depot corner. greenhouse.wbt (100 lines) is a reduced scene used by the single-subsystem test controllers, where a smaller and fully deterministic layout makes a failure easier to attribute.

2) Why the scene is generated, not authored: The decisive design choice in Phase A was to make the working scene *procedural*. A supervisor controller, world_manager.py (185 lines), rebuilds the arena at the start of every run from a fresh random seed, which it prints so that any run can be replayed:

```
seed = random.randrange(1_000_000_000)
random.seed(seed)
print(f"[world_manager] Greenhouse seed = {seed}")
```

The alternative — one hand-authored scene — is faster to build and much more dangerous. A controller developed against a fixed layout learns the layout. Waypoints get tuned until they happen to thread between two particular basins; a planner inflation radius gets reduced until one particular gap becomes passable. None of that survives contact with a different arrangement, and none of it is visible while the scene never changes. Randomising the scene makes overfitting fail loudly and early, on the developer's own machine.

3) What is randomised: Table VI lists the generation parameters. Three of them carry most of the variability:

- POT_SKIP_PROB $= 0.25$ deletes a quarter of the planting basins at random. The rows therefore have gaps, and those gaps change every run — so the planner must actually search rather than follow a memorised corridor.
- N_CRATES $\in [2, 5]$ makes even the number of mission objectives unknown at start-up, which is what forced the mission layer to be a discovery loop instead of a fixed sequence (Sec. IV-L).
- N_WORKERS $= 2$ adds moving obstacles that wander between random aisle points at 0.30 m/s. They are the only dynamic elements in Phase A, and they are what justifies re-running the mapper during navigation instead of mapping once and planning on a frozen grid.

The KEEP_CLEAR mechanism deserves a note because it is the kind of detail that is invisible until it bites. Nothing is spawned within 1.0 m of the robot's start corner $(-4.5, 4.5)$ or the depot $(4.5, -4.5)$. Without it, roughly one run in ten began with a crate spawned underneath the robot or a basin blocking the depot, and the resulting mission failure looked like a planner defect rather than a scene-generation artefact. A generator that can produce an impossible scene will eventually produce one, on the run being demonstrated.

4) The supervisor's second role: ground truth: A Webots supervisor can read the true pose of any node in the scene graph. world_manager.py uses this for two things that a robot may not do:

1) **Scoring.** It knows how many crates it created and where they are, so it can report how many the robot found, how many it delivered, and how many it missed — an objective mission score that does not depend on the robot's own beliefs.
2) **Reference data for perception.** It can answer "what is really there?" for a detection, which is how the vision module's false-positive rate was measured rather than estimated.

This separation — the robot's belief on one side, the simulator's truth on the other, compared by a third party that participates in neither — is the single most valuable idea Phase A contributed to Phase B. It reappears there, considerably expanded, as the instrumentation layer of Sec. V-G: three nodes that continuously score pose, map and throughput against Gazebo ground truth. The measurement discipline of this project starts here.

5) Coordinate conventions: The world frame is the XY plane with its origin at the arena centre, $+x$ east and $+y$ north, following REP-103 [5]. Body-frame twists use x forward, y left, ψ counter-clockwise. The occupancy grid uses image conventions — row indexes y, column indexes x, origin at the top-left — so a vertical flip is applied whenever a grid is published or drawn. Getting that flip wrong produces a map that is a perfect mirror of reality and looks entirely plausible until a path is planned on it; it is checked explicitly in Phase B's regression suite (Sec. V-H).

Table VII: Mecanum platform constants (Phase A)

Symbol	Meaning	Value
r	wheel radius	0.050 m
l_x	longitudinal centre-to-wheel	0.228 m
l_y	lateral centre-to-wheel	0.158 m
L	$l_x + l_y$	0.386 m
$\omega_{\max}$	commanded wheel ceiling	14.0 rad/s

G. Locomotion: Mecanum Kinematics and Odometry

1) The platform: The KUKA YouBot base carries four mecanum wheels [6], each fitted with a ring of passive rollers set at 45° to the wheel plane. A conventional wheel can exert force only along its rolling direction; a mecanum wheel, because its contact patch is a roller free to slide along its own axis, exerts force along a single diagonal. Four such wheels with alternating roller handedness span the full planar velocity space, so the base is *holonomic*: it can translate in any direction and rotate, independently and simultaneously.

This matters more for this application than it may appear. A greenhouse service aisle in the Phase B twin is 0.80 m wide while the base is 0.58×0.38 m; a differential-drive robot would have to perform a three-point turn to change rows, whereas the YouBot can strafe sideways out of a row without changing heading. Sec. V-C shows that this property became the basis of the escape manoeuvre after a rotating recovery was measured to be actively harmful.

2) Inverse kinematics: Let the body frame follow REP-103 [5]: x forward, y left, ψ yaw counter-clockwise. With wheel radius r, longitudinal half-spacing l_x and lateral half-spacing l_y, and writing $L = l_x + l_y$, the four wheel angular velocities $\omega_{1..4}$ that realise a commanded body twist (v_x, v_y, ω_z) are

$$\omega_1 = \tfrac{1}{r}\left(v_x + v_y + L\,\omega_z\right) \quad \text{(front left)} \tag{1}$$
$$\omega_2 = \tfrac{1}{r}\left(v_x - v_y - L\,\omega_z\right) \quad \text{(front right)} \tag{2}$$
$$\omega_3 = \tfrac{1}{r}\left(v_x - v_y + L\,\omega_z\right) \quad \text{(rear left)} \tag{3}$$
$$\omega_4 = \tfrac{1}{r}\left(v_x + v_y - L\,\omega_z\right) \quad \text{(rear right)} \tag{4}$$

with the measured platform constants of Table VII. The sign pattern is the one published in the Cyberbotics YouBot reference implementation, keyed by the *native* device index `wheel1...wheel4`. Keying by native index rather than by physical corner is a small decision that removed a whole class of bug: no remapping table exists, so no remapping table can be wrong.

3) Saturation that preserves direction: Equations (1)–(4) can demand a wheel speed the motor cannot deliver, and the YouBot ceiling is about 14.8 rad/s. The naive remedy — clipping each wheel independently — is wrong, and visibly so: clipping one wheel of a diagonal pair while leaving the other changes the *direction* of the resulting twist, so a robot asked to strafe left while turning drifts forward instead. The implementation therefore computes the peak demand and, if it exceeds the ceiling, scales *all four* wheels by the same factor:

$$\omega_i \leftarrow \omega_i \cdot \min\left(1, \frac{\omega_{\max}}{\max_j |\omega_j|}\right), \qquad i = 1 \ldots 4. \tag{5}$$

The robot then moves more slowly than asked, but in exactly the commanded direction. This is the mecanum analogue of the general principle that a saturation should preserve the shape of a command; the same principle reappears in Phase B, where the protective stop scales the translation by a factor rather than truncating one axis (Sec. V-C).

4) Forward kinematics and odometry: Odometry inverts the same relation. Given the four measured wheel increments $\Delta\theta_i$ over one control period, the body displacement is

$$\Delta x = \tfrac{r}{4}\left(\Delta\theta_1 + \Delta\theta_2 + \Delta\theta_3 + \Delta\theta_4\right) \tag{6}$$
$$\Delta y = \tfrac{r}{4}\left(\Delta\theta_1 - \Delta\theta_2 - \Delta\theta_3 + \Delta\theta_4\right) \tag{7}$$
$$\Delta\psi = \tfrac{r}{4L}\left(\Delta\theta_1 - \Delta\theta_2 + \Delta\theta_3 - \Delta\theta_4\right) \tag{8}$$

integrated into the world frame with the mid-point heading $\psi + \Delta\psi/2$ to avoid the first-order bias of using the heading at the start of the interval.

Two properties of this estimator dominate everything built on top of it, and both were measured in `odom_test`:

- **It is exact in the absence of slip.** Over short, straight motions the estimate tracks the ground-truth GPS to within a few millimetres.
- **Its error is unbounded and monotone.** Mecanum rollers slip by construction, particularly during rotation, and every slip is integrated and never forgotten. This is not a defect to be tuned away; it is the reason localisation exists, and it is why Phase B devotes an entire subsystem to bounding it (Sec. V-D).

5) Control flow: Fig. 2 gives the base controller as it runs in `base_test` and, unchanged in substance, inside the mission controller. The banding is instructive: everything that can be computed once — device handles, the constant L, putting the wheel motors into velocity mode by setting their position target to infinity — is in the initialisation band, and the main loop contains nothing but the kinematics and the saturation. The termination band exists because a Webots controller that exits without zeroing the wheel velocities leaves the robot coasting into the next run.

6) Validation: `wheel_test` commands each wheel in isolation and verifies sign and rate against its position sensor. `base_test` commands the three canonical twists — pure forward, pure strafe, pure yaw — and checks that the GPS displacement matches the commanded direction to within a tolerance. `odom_test` drives a closed rectangle and reports the loop-closure error, which is the quantity later used to calibrate the systematic component of the error in Phase B (Sec. V-D). Running the three in order takes under two minutes and was repeated after every change to the kinematics.

H. Perception: Lidar and Colour Vision

Phase A carries exactly two exteroceptive sensors, and they answer two different questions. The lidar answers *"where can*

I go?"; the camera answers *"what is that, and where is it?"*. Neither can answer the other's question, and a substantial part of the mission logic (Sec. IV-L) exists to reconcile the two.

1) The lidar: what the sensor physically is: The sensor is declared in the robot's `bodySlot` as

```
Lidar {
  translation 0 0 0.2
  name "lidar"
  horizontalResolution 360
  fieldOfView 6.28
  numberOfLayers 1
  maxRange 12
}
```

Read literally, this is a single horizontal plane, 0.20 m above the base origin, sampled at 360 bearings over a full 6.28 rad revolution, with a 12 m ceiling. Four consequences follow directly from those five numbers, and every one of them shaped a later design decision:

1) **It is a slice, not a view.** The lidar sees only what intersects one horizontal plane at one height. An obstacle below that plane — a crate lying on the floor — is invisible to it, which is precisely why crate detection had to be done in the camera and not in the lidar.
2) **Its angular resolution is $1°$.** Two returns 2 m away are 3.5 cm apart; at 5 m they are 8.7 cm apart. A gap narrower than the beam spacing at that range is simply not resolved, so the far field of a scan is systematically coarser than the near field. The mapper inherits this: a distant cell receives fewer and more widely spaced updates, and therefore converges more slowly — which is the correct behaviour, not a defect.
3) **It measures range, not free space.** A beam that returns 3 m asserts one occupied point *and* a 3 m segment of emptiness behind it. Turning that pair of assertions into a map is the entire subject of Sec. IV-I.
4) **Its origin is the base centre, not the bumper.** The sensor sits at the geometric centre of a 0.58×0.38 m body. A raw range of 0.28 m straight ahead therefore describes a point *inside* the robot's own nose. This single fact caused the most damaging defect of the whole project, in Phase B, and is dissected in Sec. V-C.

a) A common misconception, stated and corrected: A $360°$ lidar is often described as giving a robot "awareness of everything around it". It does not. It gives a *complete horizontal ring of distances at one height, with no identity attached to any of them*. The scan cannot distinguish a wall from a plant gutter from a moving worker; it cannot see above or below its own plane; it has no memory from one scan to the next; and it produces no notion of "a corridor" or "a row" — those are inventions of the mapper and the planner. Every behaviour that resembles spatial understanding in this system is software built on top of a ring of numbers.

2) The camera: intrinsics as declared:

```
DEF CAMERA_RGB Camera {
  translation 0.2 0 0.3
  rotation 0 1 0 0.5
  name "camera"
  width 160  height 120
  fieldOfView 1.2
}
```

The mount is fixed and known: 0.20 m ahead of the base centre, 0.30 m above the body slot (about 0.40 m above the floor), pitched 0.5 rad downward about the body y axis. The image is 160×120 with a horizontal field of view of 1.2 rad. From these the pinhole intrinsics follow with no calibration step at all:

$$f_x = f_y = \frac{W/2}{\tan(\theta_{\text{fov}}/2)} = \frac{80}{\tan 0.6} \approx 118.6\,\text{px}, \qquad (9)$$

with the principal point at $((W-1)/2, (H-1)/2)$. Square pixels are assumed, which is exact for a Webots camera.

3) Detection stage 1 — brightness-invariant red segmentation: Webots returns the image as a BGRA byte buffer, rows top to bottom. The first version of the detector used an absolute margin: a pixel was red if $R - G > 60$ and $R - B > 60$. It worked on the test bench and failed in the arena, because the checkerboard floor is itself a brownish red. The margin that rejected the floor also rejected any crate in shadow, where all three channels scale down together.

The fix was to threshold on *channel ratio* rather than difference:

$$\mathcal{M}(u,v) = [R > 60] \wedge [R \geq G] \wedge [R \geq B] \wedge [G < 0.33\,R] \wedge [B < 0.33\,R] \qquad (10)$$

The measured pixel statistics justify the constant. A crate pixel has $G/R \approx 0.12$; a floor pixel has $G/R \approx 0.40$. A threshold of 0.33 separates them with margin on both sides and — this is the point of the change — a ratio is invariant under illumination scaling, so the same threshold holds in shadow where an absolute margin does not. The implementation is four vectorised comparisons over the whole array:

```
limit = self.red_ratio * r
return ((r > self.value_min) & (r >= g) & (r >= b)
        & (g < limit) & (b < limit))
```

4) Detection stage 2 — connected components: The mask is grouped into blobs by 4-connectivity flood fill, implemented with an explicit stack rather than recursion, because a 160×120 mask can produce a component deep enough to exhaust the Python recursion limit. Components smaller than `min_blob_pixels` $= 4$ are discarded as noise. That threshold is deliberately low: a crate at 3 m occupies only a handful of pixels in a 160×120 image, and raising the threshold to a comfortable-looking 20 pixels made the robot blind beyond 1.5 m. Each surviving component contributes its centroid $(\bar{u}, \bar{v})$ in pixel coordinates.

5) Detection stage 3 — back-projection onto the ground plane: A single camera cannot recover depth. It can, however, recover position *given a plane constraint*, and every crate rests on the floor at a known height $z_g = 0.05$ m. The centroid is first turned into a ray in the camera frame (Webots convention: $+x$ forward, $+y$ left, $+z$ up),

$$\mathbf{d}_c = \left(1,\ -\frac{\bar{u} - c_x}{f_x},\ -\frac{\bar{v} - c_y}{f_x}\right), \tag{11}$$

then rotated into the world by the camera's world orientation, which is *composed* rather than read from the simulator:

$$\mathbf{R} = \mathbf{R}_z(\psi)\,\mathbf{R}_y(\beta), \qquad \mathbf{p} = \begin{pmatrix} x_r + d\cos\psi \\ y_r + d\sin\psi \\ h_c \end{pmatrix}, \tag{12}$$

with ψ the robot yaw taken from its own pose estimate, $\beta = 0.5\,\text{rad}$ the fixed mount pitch, $d = 0.20\,\text{m}$ the mount offset and $h_c = 0.40\,\text{m}$ the camera height. Intersecting the world-frame ray $\mathbf{d}_w = \mathbf{R}\,\mathbf{d}_c$ with the ground plane gives the scale and the estimate:

$$t = \frac{z_g - h_c}{d_{w,z}}, \qquad (x, y) = \big(p_x + t\,d_{w,x},\ p_y + t\,d_{w,y}\big). \tag{13}$$

a) Why the camera pose is composed and not read: The supervisor API could return the camera node's true world transform in a single call. Using it would have been shorter, and it would have been cheating: the estimate would then be exact regardless of how badly the robot's own localisation was drifting, and the perception error measured in `vision_test` would have been meaningless. Composing the pose from the robot's *own* estimate and a fixed, documented extrinsic keeps the whole detection chain onboard, and makes localisation error propagate into object position error exactly as it would on hardware. This is the same discipline that later separates `truth_monitor` from the robot in Phase B (Sec. V-G).

6) Three rejection gates: Equation (13) is numerically unstable near the horizon: as $d_{w,z} \to 0$ the intersection runs to infinity, so a one-pixel error in $\bar{v}$ can move the estimate by tens of metres. Three gates bound the damage, and each was added in response to an observed failure rather than anticipated:

- `min_ray_down` $= 0.12$ — reject rays not pointing down steeply enough. This is the horizon guard.
- `max_range` $= 3.0\,\text{m}$ — reject ground hits beyond the distance at which a crate is more than a few pixels wide.
- `arena_bound` $= 4.6\,\text{m}$ — reject any estimate outside the arena. A detection that lands inside a wall is wrong by construction, whatever produced it.

7) Two defects found only by testing against truth: `vision_test` spins the robot in place for 30 s and prints, once a second, the detections beside the supervisor's true crate positions. Two defects surfaced that no amount of staring at the mask would have revealed.

One-off false positives. A single frame in which a moving worker's red-brown texture passed Eq. (10) was enough to register a phantom crate, after which the mission would drive across the arena to collect nothing. The fix was *temporal confirmation*: a detection must be seen in several consecutive frames within a small spatial window before it is admitted to the crate list. The cost is a short delay before a real crate is accepted; the benefit is that uncorrelated single-frame noise is rejected outright.

Detections sitting on lidar obstacles. The remaining false positives clustered on the workers, which are both reddish and mobile. The fix was cross-modal: at discovery time, a candidate whose projected ground position falls close to a lidar return is rejected, because a crate lies below the lidar plane and therefore *cannot* produce one. The two sensors disagreeing is itself the evidence. This is the earliest appearance in the project of a principle that runs through all of Phase B: a belief supported by one sensor and contradicted by another should be dropped, not averaged.

8) What was measured: Across the `vision_test` runs the detector found every crate that entered the field of view within 3 m, and the residual position error was dominated not by the image processing but by the yaw term in Eq. (12): a 5° heading error at 2 m range displaces the estimate by 17 cm. That sensitivity is the reason Phase B devotes an entire subsystem to bounding heading drift (Sec. V-D), and it is why the Phase B fruit localiser inherits both the strength of this design — it works, cheaply, with no training data and no labelled dataset — and its weakness: it is never better than the pose that feeds it.

I. Mapping and Localisation

1) The representation: a log-odds occupancy grid: The map is a 100×100 grid covering the 10 m arena at 0.10 m resolution, following Moravec and Elfes [7]. Each cell carries not a binary state but a *log-odds* value ℓ, the logarithm of the ratio between the probability that the cell is occupied and the probability that it is free:

$$\ell(c) = \log \frac{P(c \text{ occupied})}{P(c \text{ free})}. \tag{14}$$

The reason for the logarithm is arithmetic, not aesthetic. In probability space, fusing independent observations requires a product and a renormalisation; in log-odds space it is an addition [8]. Fusing a scan therefore becomes a sweep of `+=` over an array, which is both fast and numerically untroubled. Two constants define the sensor model actually used:

`L_OCC`	added where a beam terminates	$+0.85$
`L_FREE`	added to every cell the beam crosses	-0.40
`L_CLAMP`	saturation bound on $\lvert\ell\rvert$	5.0
`L_OCC_THRESHOLD`	ℓ above which a cell is "occupied"	0.5

Three deliberate asymmetries are encoded in those four numbers.

Occupied evidence is worth more than free evidence (0.85 against 0.40). A beam that stops has made a positive measurement of a surface; a beam that passes through has only failed to find one. Making them equal produces a map that erodes thin obstacles, because a gutter edge is crossed by many more passing beams than terminating ones.

The belief is clamped. Without `L_CLAMP`, a wall seen five hundred times reaches $\ell = 400$ and needs a thousand contradicting observations to be forgotten. The map would then be unable to react to a moving worker who parks

somewhere and later leaves. The clamp at ± 5 keeps roughly ten contradicting scans sufficient to flip a cell, which is the right time constant for obstacles moving at 0.3 m/s in a 10 m arena.

The occupancy threshold is not zero but $+0.5$. A cell must accumulate net positive evidence, not merely non-negative evidence, before the planner is told to avoid it. This is the cheapest available defence against single-beam noise.

2) Integrating one scan: exact ray casting: For each of the 360 beams the mapper computes the world-frame bearing $\alpha = \psi + \left(\frac{\text{fov}}{2} - i\,\Delta\right)$, decides whether the beam terminated on a surface (`dist` finite and below `max_range`), and walks the integer line from the robot cell to the endpoint cell with Bresenham's algorithm [9]:

```
for (cc, cr) in self._bresenham(r0c, r0r, ec, er):
    if cc == ec and cr == er:
        break            # stop BEFORE the
    ↪ endpoint
    self.log_odds[cr, cc] += L_FREE
if hit:
    self.log_odds[er, ec] += L_OCC
```

The `break` is not decoration. Without it the endpoint cell receives `L_FREE` on the same pass that gives it `L_OCC`, and the wall's net evidence per scan falls from $+0.85$ to $+0.45$ — halving the convergence rate for no reason at all. It is a one-line detail that is invisible in the code review and obvious in the resulting map.

a) A rejected optimisation, and why it was rejected: The per-beam Python loop is the slowest part of Phase A. A numpy rewrite was implemented and benchmarked: sample each ray at half-cell intervals and scatter with `np.add.at`. It was measured to be *both slower and less accurate* — slower because `np.add.at` is not vectorised internally over duplicate indices, and less accurate because half-cell sampling can skip the very cell a beam grazes, eroding occupied cells along oblique walls. The exact sweep was kept, and the mapper was instead run once every `INTEGRATE_EVERY` $= 4$ control ticks, which costs nothing in map quality because the robot moves less than 4 cm in that interval. Recording a rejected optimisation is as much part of the engineering record as recording an accepted one.

b) A near-range gate: Beams shorter than `min_range` $= 0.25$ m are discarded entirely. At that distance the return is either the robot's own structure or noise, and integrating it stamps a permanent obstacle onto the cell the robot is standing in — after which the planner cannot find a path out of it.

3) From belief to plan: thresholding and inflation: The planner cannot consume log-odds. `update_grid_from_log_odds` produces the binary planning grid in two steps: threshold at `L_OCC_THRESHOLD`, then dilate every occupied cell by a disc of radius `inflation`.

Inflation is what turns a map of the *world* into a map of the *robot's configuration space* [10]. Once obstacles are grown by the robot's own radius, the planner may treat the robot as a dimensionless point, and any path it returns through free cells is collision-free for the real body. In Phase A the radius is 0.32 m, chosen to cover the YouBot half-diagonal of $\sqrt{0.29^2 + 0.19^2} = 0.347$ m with a small deliberate shortfall so that the 1.2 m aisles remain traversable.

That choice — a disc sized on the half-diagonal — is exactly the approximation that Phase B later had to abandon. A disc of 0.347 m is correct for a robot that may be rotated arbitrarily, but it is 16 cm too conservative sideways, and in the 0.80 m aisles of the Phase B greenhouse that conservatism is the difference between a passable lane and an impassable one. The rectangle model of Sec. V-C replaces it.

The dilation itself is done by shifted whole-array `OR`s rather than a per-cell neighbourhood scan:

```
for dr in range(-radius, radius + 1):
    for dc in range(-radius, radius + 1):
        if dr*dr + dc*dc > radius*radius:
            continue
        out[r_dst0:r_dst1, c_dst0:c_dst1] |= \
            occupied[r_src0:r_src1, c_src0:c_src1]
```

which is $O(\text{radius}^2)$ array operations instead of $O(\text{cells} \times \text{radius}^2)$ scalar ones — for a 100×100 grid and a 3-cell radius, 45 array `OR`s in place of 450 000 Python iterations.

4) Localisation, part 1: dead reckoning: `localization.py` first provides a GPS-free pose by integrating the mecanum forward kinematics of Sec. IV-G over the four wheel encoders. It is exact in the absence of slip and its error is unbounded and monotone in its presence. Every metre driven adds error that is never removed, which is the entire justification for what follows.

5) Localisation, part 2: likelihood-field scan matching: The corrector is a *correlative scan matcher* [11]: given a pose prior from odometry, search a small window of $(\Delta x, \Delta y, \Delta\psi)$ offsets for the one that best explains the current scan against a reference map. The score of a candidate pose is a likelihood field [8]: project the scan endpoints into the map, look up each endpoint's distance d to the nearest obstacle surface, and sum a Gaussian,

$$S(x, y, \psi) = \sum_{k \in \text{beams}} \exp\left(-\frac{d_k^2}{2\sigma^2}\right), \qquad \sigma = 0.15\,\text{m}. \quad (15)$$

Three implementation decisions carry the performance, and each of them was made after the naive version failed.

a) The distance field is seeded from surfaces, not from solids: The first version computed distance-to-nearest-*occupied-cell*. This saturates: a scan shifted bodily *inside* a large planting basin still lands every endpoint on an occupied cell, scores a perfect $d = 0$ everywhere, and the search has no gradient to climb. Real lidar only ever sees obstacle *boundaries*, so the breadth-first distance transform is seeded from occupied cells that touch free space:

```
occ = grid > 0
free_neighbor[:-1, :] |= ~occ[1:, :]   # and the
    ↪ three other shifts
return occ & free_neighbor
```

Endpoints that drift into an obstacle interior are then penalised, and the score surface acquires a genuine peak at the true pose.

b) Coarse-then-fine, with a cached beam layout: The search runs a coarse pass (± 0.30 m in steps of 0.10 m, ± 0.10 rad in steps of 0.05 rad) followed by a fine pass around the winner (± 0.06 m / 0.02 m, ± 0.04 rad / 0.02 rad). That is $7^3 = 343$ candidates plus 7^3 again, rather than the $31^3 = 29\,791$ a single fine sweep would need. Every beam is subsampled by `beam_stride` $= 6$ (60 of 360 beams), and the subsampled indices and their local bearings depend only on the scan geometry — constant across all 686 candidates — so they are computed once and cached. The score itself is fully vectorised, which measured about ten times faster than the equivalent per-beam loop.

c) A strictly-better trust gate: The corrected pose is accepted *only if it scores strictly higher than the prior*:

```
if score <= prior_score:
    return prior, prior_score
```

The reason is a failure that was observed rather than predicted. In a region of long straight walls the score surface has a broad plateau: a scan shifted along the wall explains the map exactly as well as the true pose. Without the gate the search settles on an arbitrary point of that plateau, the result is fed back into the odometry as a reset, and the next iteration starts from a slightly worse prior — a slow runaway driven entirely by ties. Comparing with $\leq$ rather than $<$ also covers the degenerate case of an uninformative scan, where every candidate scores identically and the search would otherwise adopt a window-corner offset on no evidence at all.

This single line is, in retrospect, the most important defensive decision in the localisation stack, and its Phase B descendant is the rule that a scan-matching estimate may never be trusted merely because it exists (Sec. V-D, where the homemade matcher is shown to be *worse* than the prior it was meant to improve).

6) Assembling the estimator: `LocalizationEstimator` bundles the two behind one `pose()` callable: odometry integrates every tick, and every `correct_every` $= 8$ ticks the scan matcher snaps the estimate back onto the reference map and resets the odometry to the corrected value. Between corrections the pose tracks the wheels; the drift is bounded by the correction period rather than by the distance driven. Because the interface is a single callable, the mission layer can be switched between ground-truth GPS and this estimator by changing one argument — which is how the two were compared quantitatively.

7) Frontier extraction: deciding where to look next: `exploration.py` closes the loop between the map and the mission. Rather than patrolling a fixed waypoint list, the robot drives to the boundary between known-free and unknown space [12]. Working directly on the log-odds array,

$$\text{free}(c) \iff \ell(c) < -0.5, \tag{16}$$

$$\text{unknown}(c) \iff |\ell(c)| < 0.1, \tag{17}$$

$$\text{frontier}(c) \iff \text{free}(c) \wedge \exists\, c' \in \mathcal{N}_4(c) : \text{unknown}(c'). \tag{18}$$

The three-way test is why the log-odds array is kept rather than discarded after thresholding: a binary grid cannot distinguish "observed and empty" from "never observed", and frontier exploration is exactly the algorithm that needs that distinction.

Frontier cells are clustered by 4-connectivity, clusters smaller than `min_cluster` $= 4$ cells are dropped as noise, and each surviving cluster contributes one goal. Two refinements matter:

- The goal is the frontier cell *nearest the robot*, never the cluster centroid. For a ring-shaped frontier — the normal shape early in a run — the centroid falls in the middle of already-explored space, and the robot is sent to a place it has already been.
- Cells closer than `min_dist` $= 0.6$ m are skipped, otherwise the robot twitches in place chasing a frontier it is already dissolving.

The resulting goals are ordered by a greedy nearest-first tour, and an empty list is the natural termination condition: no frontier means everything reachable has been seen. In Phase B this adaptive scheme is replaced by a deterministic three-lap boustrophedon survey (Sec. V-F) — not because frontier exploration failed, but because a fixed structured aisle layout makes a coverage pattern both simpler and time-bounded, and the report's time axis (Sec. XVII) made bounded completion time a first-class requirement.

J. Planning and Path Following

Planning and following are two different problems and are solved by two different pieces of code. The planner answers *"what sequence of free cells connects here to there?"* once, on a static snapshot of the map. The follower answers *"what should the wheels do in the next 32 ms?"* continuously, on a pose that is always slightly wrong. Conflating them — a single controller that both searches and steers — is a recurring failure mode in student robotics projects, and this project deliberately avoided it from the first commit.

1) A* on the inflated grid: The planner is A* [13] over the 8-connected 100×100 inflated grid produced in Sec. IV-I. Straight moves cost 1, diagonal moves cost $\sqrt{2}$, and the heuristic is the *octile* distance, which is the exact cost of the cheapest unobstructed 8-connected path:

$$h(a,b) = \sqrt{2}\,\min(\Delta_x, \Delta_y) + \big|\Delta_x - \Delta_y\big|, \qquad \Delta_x = |a_x - b_x|. \tag{19}$$

The choice of heuristic is not cosmetic. Manhattan distance ($\Delta_x + \Delta_y$) *over*-estimates the cost of a diagonal move — it charges 2 where the true cost is 1.414 — and an inadmissible heuristic destroys A*'s optimality guarantee, which in practice

shows up as paths that take an unnecessary dog-leg around a basin. Euclidean distance is admissible but loose on a grid that cannot move at arbitrary angles, and a loose heuristic expands more nodes than necessary. Octile is both admissible and tight for this connectivity, which is exactly the property one wants: optimal paths, minimal expansions. On the Phase A grid a typical query expands a few thousand cells and returns in a few milliseconds — fast enough to replan on every mission event rather than clinging to a stale plan.

2) Two robustness measures around the search: The textbook algorithm assumes the start and goal are free cells. In a running system, neither can be assumed, and both violations were observed within the first hour of testing.

a) Nearest-free snapping: The robot's own cell is frequently *inside* the inflation band — that is what standing next to a wall means once obstacles have been grown by the robot radius — and A⋆ started from an occupied cell returns no path at all. Likewise, a crate detected by the camera sits on the floor beside a basin, and its cell is often inflated. The planner therefore snaps both endpoints to the closest free cell by an expanding square-ring search:

```
for radius in range(1, max(rows, cols)):
    for dr in range(-radius, radius + 1):
        for dc in range(-radius, radius + 1):
            if max(abs(dr), abs(dc)) != radius:
                continue                    # ring,
    ↪ not disc
            if self._is_free(cand):
                return cand
```

The `continue` keeps the scan on the ring boundary, so cells are visited in increasing Chebyshev distance and the first free cell found is the nearest one. Snapping the goal is what makes "drive to the crate" mean "drive as close to the crate as the configuration space allows" rather than "fail".

b) Collinear smoothing: A raw A⋆ result is one waypoint per cell: a 6 m path is sixty waypoints, fifty-eight of which lie on straight segments. The follower does not benefit from them and the mission log becomes unreadable. `_smooth` keeps a cell only when the step direction changes:

```
if (dx1, dy1) != (dx2, dy2):
    out.append(cells[i])
```

Sixty waypoints typically collapse to five or six, with the geometry of the path unchanged — this is a pure compression of the representation, not a shortcut. Genuine shortcutting (line-of-sight smoothing across the grid, as in Theta⋆) was considered and rejected: it produces paths that graze the inflation boundary, and the inflation margin is the only collision guarantee the system has.

3) Following the plan: pure pursuit: The follower is pure pursuit [14], [15]. Its state is a *cursor* on the polyline, (i, t), meaning "fraction t along segment i". Each tick:

1) The cursor is advanced to the orthogonal projection of the robot onto the polyline, and *never allowed to move backwards* (`self._cursor_t = max(self._cursor_t, t)`). Without that monotonicity, a robot pushed sideways past a corner will re-acquire an earlier segment and drive the path twice.
2) A target is taken `lookahead_distance` $= 0.4$ m further along the polyline, walking segment by segment so the distance is measured *along the path*, not as a chord.
3) A velocity command toward that target is issued, braked proportionally within `brake_distance` $= 0.6$ m of the final waypoint but never below `min_speed` $= 0.12$ m/s.
4) Success is tested against the *final waypoint* and the `position_tolerance` $= 0.15$ m, not against the cursor. Testing the cursor makes arrival depend on how the path happened to be discretised.

The lookahead distance is the single tuning knob, and it trades two failure modes against each other: too short and the robot oscillates about the path, over-correcting on every tick; too long and it cuts corners, which in a greenhouse means clipping a basin. The value 0.4 m was found by bisection against the 1.2 m aisle width of the Phase A arena, and it is worth noting that the Phase B greenhouse, with 0.80 m aisles, required it to be reduced to 0.32 m — the parameter is a property of the environment, not of the algorithm.

4) The holonomic command law — and the defect hidden in it: Phase A exploits the mecanum base directly. The velocity toward the lookahead target is computed in the world frame, rotated into the body frame, and issued *together with* a proportional yaw command that softly turns the base to face the direction of travel:

$$v_x^{\text{body}} = \cos\psi\, v_x^w + \sin\psi\, v_y^w, \tag{20}$$

$$v_y^{\text{body}} = -\sin\psi\, v_x^w + \cos\psi\, v_y^w, \tag{21}$$

$$\omega_z = \text{clip}(k_\psi\, \text{wrap}(\text{atan2}(\Delta y, \Delta x) - \psi),\ \pm 1.5)\,. \tag{22}$$

This is legal, elegant and, in the Phase A arena, effective: the base translates immediately along the path while its heading catches up, so no time is lost turning, and the lidar and arm end up pointing where the robot is going. It was validated in `nav_test` and used for the whole of Phase A without complaint.

It also contains a defect that Phase A was structurally incapable of revealing, and that cost several days in Phase B. Because translation and rotation are commanded simultaneously, the body points one way and travels another — it *crabs*. Two consequences follow:

- **The robot becomes wider.** A 0.58×0.38 m body crossing an aisle at $45°$ sweeps 0.68 m of it instead of 0.38 m. In a 1.2 m Phase A aisle that is harmless. In a 0.80 m Phase B aisle it is fatal.
- **The protective stop is aimed at the wrong obstacle.** Phase B's guard tests the corridor swept *along the commanded direction*; when that direction is oblique, the test rectangle points into the gutter beside the robot instead of down the lane ahead of it, and the brake fires for something the robot was never going to reach.

The measured cost in Phase B was **46%** of mission time spent braked inside lanes the robot could have driven straight

down. The correction — turn to face the target, then translate along the body x axis scaled by $\cos$(heading error), reserving holonomic translation for the two manoeuvres that genuinely need it — is presented with its measurements in Sec. XX. It is recorded *here*, in the Phase A chapter, because that is where the defect was written, and because it is the clearest example in this project of a design that is correct in one environment and wrong in another for reasons no amount of code review would surface. Only a narrower corridor and a time measurement exposed it.

5) Validation: `planner_test` builds a grid from the supervisor, plans between fixed start/goal pairs, and verifies that the returned polyline traverses only free cells — the property that inflation is supposed to guarantee. `nav_test` then drives those plans and reports final position error against the GPS. Both are single-purpose controllers with no mission logic, which is what makes a failure in either of them attributable to one subsystem.

K. Manipulation: the Five-DOF Arm

[Section in preparation.]

L. Mission Supervision with Behaviour Trees

[Section in preparation.]

M. Phase A Results and Lessons Carried Forward

[Section in preparation.]

V. Phase B — ROS 2 / Gazebo Digital Twin: Overview

[Section in preparation.]

A. The Digital Twin: SDF World and URDF Robot

[Section in preparation.]

B. Software Architecture and the lib/ Boundary

[Section in preparation.]

C. The Protective Stop: a Single-Owner Safety Guard

1) Why containment cannot be physical: The base is driven kinematically. Gazebo's `VelocityControl` plugin imposes the commanded velocity on the model at every physics step, which means a contact response is overwritten by the next command before it can decelerate anything. Making the greenhouse walls solid therefore does not contain the robot: it produces a chassis that penetrates the glass and stays there, which is precisely what the early logs show.

This is not a simulation artefact to be apologised for. On the real machine one would never allow a collision to be the limit either — a protective device that acts only after contact is not a protective device. In both worlds the same conclusion holds: containment must act on the *command*, before it reaches the wheels, and it must be impossible to route around.

That last clause is the design requirement that the first implementation failed. `navigation_node` carried its own lidar brake, but `mission_node` takes the base over during visual alignment and picking and published directly to `/cmd_vel`, bypassing the brake entirely. The failure is legible in one log line: `Aligned at (-4.02, +0.84)`. In an 0.80 m aisle whose gutter edge sits at $y = 1.0$ m, a 0.38 m-wide base centred at $y = 0.84$ already overlaps the gutter. The robot did not drive into the gutter despite the brake; it drove in through a publisher the brake never saw.

The correction is structural rather than parametric. Every publisher now writes `/cmd_vel_raw`; `safety_node` is the only node that writes `/cmd_vel`. There is one place where safety is enforced, and no way to reach the wheels without passing through it.

2) Three layers, each able to veto:

	Layer	What it vetoes
1	Lidar	Anything inside the region the footprint will sweep along the commanded direction cancels the *translation*. Rotation is preserved, so the robot can always turn away and the planner can reroute.
2	Fence	Beyond the arena bounds, the only command allowed is the one driving back inside.
3	Timeout	No command for `cmd_timeout` seconds stops the base. A publisher that dies must never leave the robot coasting.

Layer 2 checks all four corners of the 0.58×0.38 m footprint against the wall faces, rotated by the current heading, rather than bounding the centre. Bounding the centre is what let the chassis clip the glass on a diagonal: at 45 ° a corner reaches 0.35 m from the centre, not the 0.19 m a side-on estimate suggests.

3) The swept region is geometry, not a constant: The first two versions of layer 1 were both wrong, and the way in which each was wrong is more instructive than the final answer.

A cone is the wrong shape. The original test took the raw beam range inside a ± 0.55 rad cone about the direction of travel. A cone widens with distance — at 0.5 m it already reaches 0.28 m sideways — so a beam grazing a gutter the robot is legitimately driving *past* comes back short and brakes it in open corridor. Worse, the range along an oblique beam is not the distance to the obstacle along the path, so the test also overstates how soon the robot arrives.

A constant band is the wrong width. The cone was replaced by a swept rectangle of half-width 0.24 m, and a single number cannot be right here: the base does not turn to face where it is going, so the width of its shadow depends entirely on the direction of travel. Projecting the rectangle onto the axis normal to the motion gives

$$w(\theta) = a\,|\sin\theta| + b\,|\cos\theta|, \tag{23}$$

with $a = 0.29$ m the half-length and $b = 0.19$ m the half-width. That is 0.19 m straight ahead, 0.29 m straight sideways and 0.34 m on the diagonal. The constant 0.24 m was therefore too wide in one direction and too narrow in another, and each error produced its own symptom.

Too wide going forward costs margin the aisle does not have. An inner aisle is 0.80 m across and the base is 0.38 m, leaving 0.21 m per side; 5 cm of phantom band is a quarter of

that. Evaluated against the real gutter geometry, a base offset 0.18 m from the aisle centreline — body edge still 3 cm clear, travelling exactly parallel to the gutter, closing on nothing — returns zero clearance and the guard holds translation. That is the `Translation held` that appears beside the gutters, and it is the brake firing for a wall the robot is merely driving alongside.

Too narrow going sideways leaves part of the moving footprint untested. Visual alignment strafes: travelling along its own y axis the base sweeps 0.29 m fore and aft while the guard was watching 0.24 m.

4) Clearance as a ray–box entry time: Correcting the width alone would still have left a third error. The clearance was computed as the distance along the path minus $\rho(\theta)$, the reach of the footprint along the *centreline*. But the part of a rectangle that arrives first is a corner. At 45 ° the centreline reach is 0.269 m while the leading corner is already 0.339 m out, so the guard credited itself 7 cm it did not have — in the direction in which it crosses an 0.80 m aisle.

The exact formulation removes the band and the constant reach together. For a body-frame return $\mathbf{p}$ and a unit direction $\mathbf{u}$, the distance the base may travel before touching that point is the smallest $t \geq 0$ for which $\mathbf{p} - t\,\mathbf{u}$ lies inside the footprint rectangle, i.e. the entry time of the ray $\mathbf{p} - t\mathbf{u}$ into the box $[-a, a] \times [-b, b]$:

$$t^\star = \max_{i \in \{x,y\}} \min\left(\frac{p_i - h_i}{u_i},\ \frac{p_i + h_i}{u_i} \right), \tag{24}$$

valid when it does not exceed the corresponding exit time, with $h_x = a + m$, $h_y = b + m$ and m the safety margin. The clearance is the minimum of $t^\star$ over all returns. The margin is now what its name says — 0.02 m for wheels standing proud of the chassis — rather than a compromise between two directions in which the geometry disagrees.

Equation (24) was validated against a brute-force simulation that translates the rectangle in 0.1 mm steps until it contains the point: over 20 000 random point/direction pairs the largest disagreement was 0.1 mm, the resolution of the reference itself.

Table VIII gives the effect on the real gutter geometry.

Table VIII: Guard verdict before and after, evaluated on the greenhouse gutters. *HELD* means translation cancelled.

Situation	Constant band	Exact
Centred in the aisle, straight	clear	clear
0.18 m off centre, parallel	**HELD**	**clear**
Centred, 16 ° heading error	**slowed**	**clear**
Gutter end, turning out 30 °	**HELD**	**clear**
Gutter end, turning out 45 °	**HELD**	**slowed**
Gutter end, turning out 60 °	**HELD**	**slowed**

5) Flank containment: Braking only on the commanded direction misses the way the robot actually entered the gutters. During visual alignment the mission commands v_x only; a small heading error turns that into lateral drift that no forward-looking test observes, because nothing is ever *commanded* sideways. A separate rule therefore maintains a minimum clearance on both flanks (`side_min` = 0.10 m from the flank edge) and pushes back at up to `side_push` = 0.12 m/s. Being sensor-based, it holds the robot in any corridor rather than in this greenhouse specifically.

6) One geometry, one implementation: Commissioning stage 6 measures the minimum clearance the robot actually achieves during autonomous running, and it is that measurement which decides whether the protective distance computed from ISO 13855 is respected in practice. It originally carried its own copy of the corridor geometry, including the same 0.24 m constant.

A certification tool that measures clearance differently from the guard that enforces it will certify a distance the robot never kept. Stage 6 now imports the same function `safety_node` brakes on. This is stated here rather than in the commissioning section because it is a safety property, not a housekeeping one: the number in the report and the number in the brake are the same number by construction.

7) Observability: The guard publishes `/safety_override` whenever it is overriding rather than passing a command through. Without it, a publisher commanding into an override simply fights it — the field log shows twenty seconds of `Outside the arena` at three-second intervals while the mission kept creeping after a berry, each side unaware of the other. The guard also reports *which* return stopped it, in body coordinates: a protective stop that cannot say what it stopped for cannot be diagnosed from a log, and two debugging sessions were spent on the message `Obstacle within 0.12 m` before that was understood.

D. Localisation: Odometry Calibration and a Homemade SLAM

[Section in preparation.]

E. Fruit Detection and Camera-to-Map Localisation

[Section in preparation.]

F. Mission: Coverage Survey then Harvest

[Section in preparation.]

G. Instrumentation: Measuring Pose, Map and Time

[Section in preparation.]

H. The Pre-Flight Regression Suite

1) The cost that motivated it: A Gazebo run of this mission takes between twenty and thirty minutes. A run that dies in its first second because of a shadowed import, or spends its entire survey fighting a mis-set fence, costs half an hour and yields a log nobody wants to read. Several such runs were lost before the obvious response was adopted: make the cheap failures fail cheaply.

`scripts/check_regressions.py` is pure Python — no ROS, no Gazebo, no build — and the whole suite executes in about one second. It is run before a simulation, not after.

2) The rule: nothing hypothetical: The suite is not a test suite in the usual sense, and the distinction matters. It contains no check written because a check seemed prudent. Every one of the **341** encodes a failure this project actually suffered, and each carries in its failure message the run it comes from, what was expected, what was found, and where to look. A check that has never caught anything is a check whose cost is paid every day for a benefit that has never arrived; a check derived from a lost afternoon pays for itself the first time it fires.

The consequence is that the file reads as an incident history. Table IX groups it.

Table IX: The regression families, and the failure each was written after.

Family	The failure it encodes
XML well-formedness	A double hyphen inside an XML comment is illegal, and the resulting `ExpatError` names a line in a generated string, not in the file that was edited. Two debugging sessions.
Shadowed messages	A local module named like a ROS message package silently shadows it, and the node dies on the first import at run time, not at build time.
Single fence	Two nodes both containing the robot, with different bounds, fight each other at the boundary.
Brake is not an override	A protective stop that a publisher can bypass is not a protective stop (Sec. V-C).
Bumper clearance	`stop_distance` was measured from the lidar at the centre of a 0.58 m base, so the brake fired when the wall was 1 cm inside the robot's own nose.
Corridor geometry	The swept band and the footprint reach must be computed rather than assumed, and the guard, the escape and the commissioning tool must all use the same computation.
Parameters match code	A parameter renamed in one node and not in the other, defaulting silently.
Fruit map	The corroboration rule: two passes, a viewpoint baseline, and a gate that fails open with a warning rather than closed in silence.
Alignment state feedback	Closed-loop poles verified against a brute-force simulation, after a sign error put a pole at 1.59.
Label source	Training labels must come from the ground-truth catalogue, never from the detector being replaced.
Real training data	The dataset importer must exist and must refuse to guess a class mapping.
Document builds	Both LaTeX documents must actually compile.

3) Two checks worth describing in full: Most of the suite tests a property of the source text — a topic name, a parameter default, an import. Two do real computation, and those are the ones that have caught genuine mathematical errors.

Closed-loop pole placement. The visual-alignment controller is a discrete state feedback whose gains are derived analytically. An input sign error placed a closed-loop pole at 1.59: an unstable law that would have driven the base into the plant row. The check computes the eigenvalues of $A - BR$ and asserts they lie inside the unit circle, and it caught the error before the robot ran. A second version of the same defect then appeared in the reporting function, which had kept the pre-fix signs and cheerfully described a converging loop as unstable; the check now cross-validates the analytic poles against a brute-force simulation of the recursion, so the two cannot disagree again.

Clearance against brute force. Equation (24) is verified by translating the footprint rectangle in 0.1 mm steps until it contains the point, over hundreds of point and direction pairs, and asserting agreement to within the reference's own resolution. An analytic geometry result that is only inspected by eye is an assumption; one checked against a simulation of the thing itself is a result.

4) What the suite cannot do, and the lesson from that: The suite's limits were established the expensive way. An earlier LaTeX checker counted braces and verified that every `begin` had a matching `end`. It passed, and the document did not compile: `\degree` is provided by `gensymb` or `siunitx`, neither of which was loaded, and no amount of brace counting can see an undefined control sequence. The document had in fact never been compiled at all.

The rule extracted from that, and now applied throughout, is that a proxy for an outcome is not the outcome. The check no longer inspects the LaTeX source; it runs `pdflatex` and requires exit status zero. Likewise the perception checks do not assert that a threshold looks reasonable — they instantiate the camera model and verify that a berry at a known position projects where the ground truth says it does.

The same principle bounds what the suite is worth. It proves the code still means what it meant yesterday. It cannot prove the robot works, and no static check ever will; that is what Sec. XVII and the commissioning stages are for. Its claim is narrower and still worth having: of the failure modes this project has already paid for, none can return silently.

VI. Phase C — The Connected Measurement Node

A. A different question

Phases A and B answered one question: *how does a robot operate autonomously in a space it does not know?* The apparatus that question demands — occupancy mapping, SLAM, A* over a grid that might contain anything, exploration policies — was built, measured, and is reported in Sections IV–V-H.

Phase C answers a different one: *how does a fleet operator obtain data from a greenhouse and know it is genuine?*

The two questions are not versions of each other, and the second is not a reduction of the first. Its difficulty is not in the robot at all. A greenhouse under management is surveyed, its rows are catalogued, and the positions to be sampled are decided by an agronomist long before a robot is bought. What is genuinely hard is everything around the driving: an order that only the operator can have issued, a reading that arrives unaltered, a record that says when it was taken as well as what it said, and a system that refuses bad data loudly instead of filing it quietly. Those are properties of a distributed system, and Phases A and B had none of them — the harvester was a closed loop with no outside, no identity, and no adversary.

B. What was delivered

Phase C is a two-process system with a broker between them, and the separation is the point rather than an implementation detail:

- a **mobile node** — the Phase B YouBot, re-equipped with a downward alignment camera on a sensor boom — which holds a private key, verifies every order it receives, drives to painted reference marks, records the plant environment, photographs the plant, and returns a sealed report;
- a **Cloud** — a separate process, intended for a separate machine — which holds the other private key, signs and issues orders, opens and validates reports, files them, and serves an operator dashboard and a live console.

Neither knows the other's address. Both connect to an MQTT broker, and either can restart without the other noticing, which is the property that makes "the robot is a node on a network" true rather than decorative (Sec. XII).

C. What is measured, and by what

The robot reports five environmental quantities per station — temperature, relative humidity, illuminance, CO_2 and soil pH — together with its own pose and velocity at the instant of reading.

These readings are synthesised, and the system says so out loud. Gazebo models no soil chemistry and no CO_2 diffusion; there is nothing in the simulator to sample. They are generated by `agri/sensors.py` as a smooth spatial field with a diurnal component and four deliberately planted anomalies, and the robot node prints this at startup on every run:

```
agri_robot youbot-01: keys in .../keys, readings
    ↪ are
SYNTHESISED by agri.sensors (Gazebo has no probes)
```

The honest description of what Phase C validates is therefore: the *chain* is real — the driving, the alignment, the encoding, the cryptography, the transport, the storage, the refusals — and the *numbers entering it* are a stand-in for a probe that does not exist in simulation. Substituting a real sensor changes one function and nothing else, which is exactly why that function is isolated.

The anomalies exist so the chain can be shown to carry information and not merely bytes: a dry patch at `P2,4`, elevated CO_2 at `P3,7`, and a pH excursion at `P1,2`. Each survives the QR encoding, the seal, the transport and the store, and appears flagged on the dashboard — which is visible in the campaign log of Sec. XVII, where `P2,4R` arrives at **47.8%** humidity against a **≈70%** field, and `P3,7R` at **1070 ppm** against **≈550 ppm**.

D. Reading order

The remaining Phase C sections follow the path of one measurement, from the coordinate that defines where it is taken to the check that proves it arrived intact:

- Sec. VII — the 48 stations, and why the world is generated from them rather than described twice.
- Sec. VIII — deterministic navigation, and the case for deleting the planner.
- Sec. IX — closing the last centimetres on a painted mark with a downward camera.
- Sec. X — the reading, the QR code, the photograph.
- Sec. XI — what is signed, what is encrypted, and why those are different questions.
- Sec. XII — the MQTT contract and the multi-node topic layout.
- Sec. XIII — the Cloud, the store, the dashboard, and the three timestamps a reading carries.
- Sec. XIV — route optimisation and the energy budget.
- Sec. XV — the 384-check pre-flight suite.

VII. The Station Catalogue and the Generated World

A. Forty-eight stations

The Phase B greenhouse (Sec. V-A) is 9.90×4.90 m internally, with three planting gutters 0.40 m wide centred at $y = -1.20$, 0.00 and $+1.20$ m, and eight plants per row spaced 0.90 m apart from $x = -3.15$ m. Phase C adds a *measurement station* on each side of each plant: 3 rows $\times$ 8 plants $\times$ 2 sides $= 48$, labelled `Pi,jR` and `Pi,jL`.

A station is a point on the floor where the robot's sensor head must stand, offset 0.55 m across the row from the plant centre. The label grammar is deliberately the one an agronomist would speak: `P2,5` names a *plant* and expands to both of its stations; `P2,5R` names one side. Expansion happens on the Cloud, before the order is signed, so the robot never interprets a wildcard — a robot that expanded `ALL` itself would silently disagree with the Cloud the day the greenhouse gained a row.

B. The decision that shapes everything else

Three programs need these coordinates and must not disagree: the world generator that paints the marks, the robot that drives onto them, and the dashboard that draws them.

The obvious construction is to type the numbers into the world file and again into the robot. That is how a mark ends up painted at $y = -1.70$ and driven to at $y = -1.65$, with the robot then reporting that it is "on" a mark it is 5 cm away from and nothing anywhere to catch it. Phase C therefore inverts the dependency: the numbers exist once, in `agri/catalogue.py`, and **the greenhouse is generated from them**.

```
python3 -m agri.world.make_world   # reads the
    ↪ Phase B scene,
                                   # paints 48
    ↪ marks from the catalogue
python3 -m agri.world.make_robot   # adds the boom
    ↪  and floor camera
```

The regression suite closes the loop in both directions: it reads the plant positions back out of the Phase B world and asserts the catalogue still agrees with them, and it parses the generated SDF and asserts every painted mark is where the

catalogue says. Neither the greenhouse nor the robot can drift from the map without a check going red (Sec. XV).

The generator also refuses to run twice over its own output, because re-painting an already-painted world produces 96 marks and a scene that looks subtly wrong rather than obviously broken.

C. Two geometric constraints that fixed the numbers

Two constants were not chosen by preference. They are reported here because both were got wrong first, and because each is the kind of value that looks arbitrary until the reason is written down.

a) Station reach, 0.55 m: An inner aisle is only 0.80 m wide between two gutter edges. The natural design — turn the base to face the plant — points the chassis' long axis across that aisle, putting 0.29 m of half-length toward the gutter instead of 0.19 m of half-width, and the two stations sharing an inner aisle then cannot both fit with usable margin. The first attempt left 1 cm.

Phase C keeps the base *along* the aisle and lets the existing pan head look sideways, which is what the head is for. That converts 0.29 into 0.19 and buys back 0.20 m. At a reach of 0.55 m, the tightest of all 48 stations (`P2,1R`) then clears every gutter and wall by **0.16 m**, and the suite asserts that bound on every run.

b) Cross arm, 0.04 m: The price of the above is that the two stations sharing an inner aisle sit only 0.10 m apart, at the same x. Their y-arms therefore lie on one line. Any arm longer than 0.05 m makes the two painted crosses touch, and a blob detector then sees *one* marker straddling both — it would centre the robot on the midpoint, 0.05 m from either station, and file the visit under whichever label it guessed.

0.04 m leaves a 0.02 m gap, about ten pixels for the floor camera. An earlier version used 0.09 m and carried a comment asserting the crosses did not merge; rendering the pair and running the detector over it is what showed the comment was wrong. The suite now renders adjacent pairs and asserts the nearer mark wins at three different offsets.

D. Why the marks are painted at all

0.10 m is inside the error a drifting odometric estimate accumulates in a single lap of this greenhouse. Driving to a station on odometry alone is therefore not merely imprecise — it is capable of parking the robot closer to the neighbouring station than to the one it is about to file a reading under, which corrupts the *label* rather than the value. The painted marks exist so that the final approach can be closed visually against a physical feature of the building, which is the subject of Sec. IX.

VIII. Deterministic Navigation

A. Autonomous, but not exploratory

This section states plainly what Phase C navigation is, because the distinction is easy to lose and a reader may reasonably assume that removing SLAM removed autonomy.

Table X: The free bands, derived from the catalogue. Each is an interval of the robot centre line in y; every station lies in exactly one.

band	y interval (m)	stations	
0	$[-2.45, -1.40]$	`P1,*R`	8
1	$[-1.00, -0.20]$	`P1,*L, P2,*R`	16
2	$[+0.20, +1.00]$	`P2,*L, P3,*R`	16
3	$[+1.40, +2.45]$	`P3,*L`	8

The robot in Phase C is fully autonomous in the control sense: it receives a list of station labels and nothing else, and it decides its own route, drives itself in closed loop, brakes on its own sensor evidence, and corrects its own final position against what it sees. There is no teleoperation and no trajectory replay.

What it does *not* do is discover. It never builds a map, never decides where to go next based on what it found, and never searches for a path. It cannot, because the question does not arise: the map is a catalogue that predates the robot (Sec. VII), and the order names the stations explicitly. The autonomy that remains is *deterministic* rather than *exploratory*, and the difference is worth naming precisely rather than describing as "less".

B. The free space, decomposed once

The greenhouse free space is not a general planning problem. It is four corridors joined at both ends, and `agri/aisles.py` derives that decomposition from the wall and gutter coordinates rather than declaring it:

At each end, past the 8.0 m gutters, lies a *headland* that joins all four bands. A route is therefore at most three legs — out to a headland, across to the target band, in to the station — and choosing between the two headlands is a comparison of two numbers, not a search:

```
def route(sx, sy, tx, ty):
    here, there = band(sy), band(ty)
    if here is not None and here == there:
        return [(tx, ty)]                  # one
    ↪ leg
    end = min(HEADLANDS,
              key=lambda e: abs(sx - e) + abs(e -
    ↪ tx))
    return [(end, sy), (end, ty), (tx, ty)]
```

a) Why no planner: A* over an occupancy grid would spend a millisecond rediscovering Table X, and would then be free to discover something *else* on a day when the grid was subtly wrong. That is not a hypothetical: it is what Phase A did in a 0.16 m clearance (Sec. IV-J). A closed-form route cannot produce a novel answer, which in a corridor with 0.16 m of margin is a feature.

The cost of that choice is that the geometry must be right, so it is verified exhaustively rather than argued: the suite sweeps **all 2 256 station-to-station routes**, plus every route out of the dock, and asserts the robot's rectangle never comes within 0.08 m of a gutter or a wall on any leg of any of them. That runs in about a second and needs nothing installed.

C. The asymmetry that a symmetric model hides

The two headlands are **not** at symmetric coordinates: $x_{\text{west}} =$ **-4.08** m and $x_{\text{east}} =$ **+4.87** m. The robot is not symmetric — a 0.50 m sensor boom points $+x$ and 0.29 m of chassis trails behind — so at the west end the sensor point leads and must stop short of the gutters, while at the east end the sensor point goes deep and the *tail* must clear them. The footprint ends up centred in the 0.95 m gap either way, with 0.08 m of margin at each end.

An earlier version used one number for both. Going east, that left the chassis lying alongside the gutters while the robot strafed across the greenhouse — straight through them. **Nothing would have reported it.** This robot carries no collision geometry, so driving through a gutter produces no contact, no warning and no complaint, only a recording that cannot be shown to anyone. It was caught by the clearance sweep above, which is the entire reason that function exists.

D. The brake, and what it must ignore

The route is geometric; the lidar is not. During every leg the robot tests each return against the corridor it is about to sweep. Two design choices carry the section.

a) A corridor, not a cone: A $\pm 35^\circ$ cone was the first version, and it stopped the robot dead in every aisle: at 0.35 m of lateral clearance, a gutter wall the robot is driving *parallel* to sits well inside a 35° cone, so the brake fired on the very thing the route was designed to pass. A corridor asks the only question that matters — is this in the way? — and a wall you are driving alongside is not in the way.

The corridor's edges are the extreme projections of the robot's own four corners onto the axis across the direction of travel. For a symmetric robot a half-width would do; with a 0.50 m boom in front and 0.29 m of chassis behind, a *strafing* robot sweeps 0.79 m and not centred on the base.

b) The building is not an obstacle: The brake exists for things that should not be there. The walls and the gutters should be there, they are in the catalogue, and the sweep above has already proved every route clears them. Braking for them is not caution; it is refusing to move — and it produced two failures that looked entirely different:

- *Every leg to a headland stopped.* The robot must back its tail to within 0.08 m of the end wall to fit the 0.95 m gap; the brake wanted 0.35 m of clear floor and refused 0.27 m short. The mission logged "something is in the aisle". Something was: the building.
- *A leg that shifts sideways while advancing* — 0.10 m across while travelling 0.66 m along, which is exactly what moving between the two stations of an inner aisle looks like — tilts the swept corridor by 8.6° and swings it onto the gutter the robot is driving *beside*. Same wall, different geometry, same wrong answer.

Both are resolved by projecting each return into world coordinates and discarding it when it falls within 0.12 m of catalogued structure. A return *inside* the robot's own outline is discarded too: the lidar sits within the chassis and sees the camera pedestal and the mast at 0.12 m and 0.22 m ahead, which a raw range test brakes for permanently, from the first metre.

The suite pins both behaviours from opposite sides — a gutter being driven past does not stop the robot, an object genuinely ahead does, and no station stands on a point the brake would ignore — so neither fix can be undone by loosening the other.

IX. Closing the Last Centimetres

A. Why odometry is not enough here

Sec. VII established the constraint: two stations sharing an inner aisle are 0.10 m apart. Sec. V-D established the measurement: uncalibrated odometry on this platform diverges past 10 m over a mission, and even calibrated it holds only **0.20 m**.

0.20 m is twice the station spacing. The failure this produces is not an imprecise reading — it is a *correctly measured value filed under the wrong label*, which is worse, because nothing downstream can detect it. A campaign so corrupted looks complete.

B. The sensor point is the camera, not the base

Phase C adds a downward camera on a short boom over the front bumper, and makes *that* point the one a station names.

A robot that has a body cannot see the floor under its own middle — the chassis is in the way — so a mark under the centre can be driven to but never verified. Putting the reference point under a downward camera makes "the mark is in the middle of the picture" and "the sensor is on the mark" the same statement, with no lever arm whose sign can be got wrong.

0.50 m is the smallest offset that also keeps the camera mast out of the downward view cone: the mast reaches $x = 0.27$ m at 0.45 m height, and a camera at 0.50 m sees the floor from $x = 0.27$ m outwards, so no ray crosses it. Two consequences follow and are stated once: the base centre stands 0.50 m back along the aisle from the station, and **every pose in every report is the sensor point**, not `base_link`.

C. Detecting a cross, and refusing a blob

The detector (`agri/vision.py`) is a red-dominance threshold followed by a connected-component pass, with three refusals that matter more than the detection itself:

- **A solid blob is not a cross.** A cross fills ≈**44%** of its own bounding box; a filled square fills 100%. Rejecting on fill ratio stops the robot from centring on a red object that is not a station mark. The accepted band is deliberately wide, because the exact ratio moves with the arm and thickness constants, and a detector that needed retuning whenever the paint changed would be retuned wrong.
- **A clipped cross is refused, not averaged.** A mark half out of frame has a centroid that is not its centre. Averaging it yields a confident correction in the wrong direction.
- **The nearer of two marks wins.** With a neighbour 0.10 m away the frame can contain both. The suite asserts the nearer

one is selected at three different offsets, including one where the robot starts closer to the neighbour.

The threshold itself needs no tuning: the aisle paint is dark green $(38, 64, 54)$ and the gutters are near white, so a factor-1.6 dominance test separates the marker from both by a wide margin. This is a consequence of the world being generated (Sec. VII) — the detector and the paint come from the same file.

D. The correction is bounded, and the bound is not a preference

The visual correction is capped at **0.04 m** per station.

The reasoning is the same 0.10 m that drove the arm length: a correction larger than 0.05 m could leave the robot nearer the *neighbouring* station than the one it is about to file under. The Cloud would then reject the report — correctly — for a reason with nothing to do with the plant. 0.04 m keeps a clear margin under that bound and is still four times the residual the odometric approach leaves behind.

The robot makes at most four correction attempts, stopping when the residual falls below 0.008 m.

E. What happens when the camera cannot see

The mark can be missing, occluded, or outside the frame. Phase C does not treat that as a mission failure and does not silently pretend it did not happen. The robot parks on odometry alone, says so in the log, and **increments a counter that travels with the mission report**:

```
mission 788081147cfd finished in 601 s, floor
    ↪ camera
used at 48 station(s), unavailable at 0
```

That line is the honest form of the accuracy claim. Rather than asserting a parking error the simulator cannot independently confirm, the system reports *how many of the visits were closed visually* and lets the reader judge. In the reference campaign of Sec. XVII the camera resolved its mark at **48 of 48** stations, and at **52 of 52** across the full session including the four single-station re-measurements.

Every report additionally carries its own `parking_error_m`, and the Cloud checks it against the catalogue independently of what the robot believed — so a visit filed under the wrong label is detectable after the fact, from the stored record alone.

X. The Measurement and Its Encodings

A. What one visit produces

A visit yields three artefacts, and they are deliberately redundant:

1) the five readings as numbers, with the station label, a UTC timestamp, and the robot's pose and velocity;
2) the same content as a **QR code image**;
3) a **photograph** of the plant, taken by the pan camera turned toward the row.

B. The robot reports on itself, not only on the plant

Every measurement carries the robot's own pose and its three velocity components at the instant of reading. This is not telemetry padding.

A reading taken while the platform is still rolling is a reading taken *somewhere between two places*. Because the velocity travels inside the sealed payload and inside the QR code, that condition is detectable afterwards, from the archived record, by someone who was not present. The velocity is read *after* the sensors rather than before, for the same reason: it is the speed at the moment of the reading that qualifies the reading.

The speed is transmitted as the magnitude alongside the components, and the suite asserts it really is the magnitude and not a component — a substitution that would look plausible in every log and be wrong for every strafing move.

C. The QR code, and why it is not decoration

The payload is a single pipe-separated line, human-legible when scanned:

```
AGRI1|P1,1R|2026-08-08T19:23:27Z|t=21.4|rh=76.2
|lux=19272|co2=552|ph=6.84|x=-3.146|y=-1.745
|yaw=180.0|vx=-0.003|vy=0.007|w=0.000
```

128 characters, which a phone camera reads off a screen and returns as the original numbers. Three properties are asserted by the suite over all 48 station codes:

- every code encodes and decodes again;
- the decoded text round-trips through the JSON *character for character*;
- a payload from another system is refused rather than misparsed — `HELLO|P2,5R` is rejected with "not an AGRI1 payload", as is an unknown quantity key.

The Cloud re-reads every QR image it receives and checks the decoded numbers against the ones that travelled beside it in the JSON. This is the project's only *end-to-end* content check: it would catch a corrupted image, a mismatched pairing, and an encoder that silently rounded. Two decoders are installed because OpenCV fails to locate 2 of the 48 station codes while zxing-cpp reads all 48; either alone works, and the tests run against both.

D. The photograph

The pan head turns toward the plant using the station's stored bearing, so the same station works whichever way along the aisle the robot is travelling — storing a fixed robot yaw instead would silently assume a direction and make half the visits look the wrong way. The suite asserts the head pans $+90°$ driving east at an `R` station, $-90°$ driving west at the same one, and the opposite for `L`.

The image is JPEG-compressed and travels base64-encoded inside the sealed envelope. The Cloud writes it back out as a file and keeps only the path in the record (Sec. XIII).

Table XI: The planted anomalies. Each is asserted by the suite to stand out from its neighbourhood, so a change to the field cannot silently flatten the signal the dashboard exists to show.

stations	quantity	offset
`P2,4R` / `P2,4L`	humidity	−22.0 / −20.0 %RH
`P3,7R` / `P3,7L`	CO_2	+480 / +455 ppm
`P1,2R` / `P1,2L`	pH	+1.35 / +1.30

E. The synthesised field

As stated in Sec. VI, the five readings are generated rather than sensed. The field is not noise: it is smooth in space, so neighbouring plants correlate; it is deterministic, so the same station reads the same twice; it carries a diurnal component driven by simulator time; and it contains the three planted anomalies of Table XI.

Both stations of an affected plant are perturbed, by slightly different amounts. A single-sided anomaly would be indistinguishable from a sensor fault, and identical values on both sides would be indistinguishable from a constant — neither would exercise the comparison the dashboard invites the operator to make.

XI. Authenticity and Confidentiality

A. Two different questions

Habit says encrypt everything. Resisting that habit is what makes this part of the design legible, so the two directions are treated separately and the reasoning is written down.

a) Cloud → robot: signed, not encrypted: A request says "measure `P2,5R`". That is not a secret. But issuing one drives a robot around a greenhouse, and anyone who can reach the broker can publish on the topic. The property that matters is therefore *authenticity*, not confidentiality. Requests carry an ECDSA-P256 signature over the canonical JSON; the robot refuses an unsigned request and refuses one signed by the wrong key.

b) Robot → Cloud: sealed and signed: A report contains the agronomic state of a commercial crop and a photograph of it. It needs confidentiality *and* origin, and gets both.

B. The envelope

Reports are sealed with ECIES over SECP256R1: an ephemeral key pair per message, ECDH against the Cloud's public key, HKDF-SHA256 to derive the content key, and AES-256-GCM for the payload. The ciphertext is then signed with the robot's long-term ECDSA-P256 key. The complete wire message is:

```
{"schema":"agri-cloud/v1", "kind":"sealed",
 "robot":"youbot-01", "request_id":"...",
 "sent_at":"2026-08-08T19:23:27Z",
 "sealed": {
   "alg":  "ECIES-P256-HKDF-SHA256-AES256GCM",
   "epk":  "-----BEGIN PUBLIC KEY-----...",
   "nonce":"UgoQGAyNp7LauuRB",
   "ciphertext":"5k2ZWwLpBQVmV61fDxBg39Xy...",
   "sig":  "MEUCIGmZLjCjPHsyhdx97dGKKhmk..." }}
```

Table XII: Cryptographic properties asserted on every run. The refusals matter more than the round trip: a system that accepts everything also round-trips perfectly.

property	verdict
a sealed payload round-trips	**accepted**
the plaintext is absent from the envelope	**confirmed**
the wrong recipient key cannot open it	**refused**
a forged sender is rejected	**refused**
stripping the signature does not bypass the check	**refused**
one flipped bit is detected	**refused**
the same plaintext seals differently every time	**confirmed**
a rewritten target breaks the request signature	**refused**

The routing metadata — who sent it, for which request, when — stays in clear, because the broker and the operator need it to route and to diagnose, and none of it is the measurement. The readings, the pose, the QR image and the photograph are all inside the ciphertext.

C. Sign-then-encrypt, or encrypt-then-sign

The signature is over the *ciphertext*. This is deliberate: it lets the Cloud reject a forged message without first performing a private-key operation on attacker-chosen input, and it means a tampered ciphertext is detected by the cheap check rather than by AES-GCM's authentication tag alone.

The suite pins six properties of this construction, and four of them are refusals:

The third-from-last matters more than it looks: an ephemeral key per message means two identical readings produce two unrelated ciphertexts, so an observer cannot tell that a station was measured twice with the same result.

D. Key custody, and the refusal that protects it

The Cloud is meant to run on a different machine from the robot. That intent is enforced rather than documented:

- An **empty** key directory is bootstrapped with both pairs, so a first run on one machine works with no ceremony.
- A **populated** key directory is never bootstrapped behind the operator's back.
- **Neither side ever generates the other side's key.** The Cloud will not invent a robot identity; the robot will not invent a Cloud identity.

That last rule is the one that earns its place. Without it, a Cloud started on a fresh machine with a forgotten key copy would generate a robot public key of its own, and then reject every report from the real robot — with a valid signature failure, forever, and no indication that the cause was a missing file. Instead it refuses to start and names the file:

```
/path/forgot/cloud_public.pem is missing.
```

The suite exercises the whole two-machine story: bootstrap an empty directory, copy only the public keys to a second directory, start a robot there, and assert that it runs *and* that it does not hold the Cloud's private key.

Table XIII: The topic contract. Robot-to-Cloud topics are namespaced per node; the Cloud subscribes with MQTT wildcards.

topic	direction	payload
`/agri/v1/request`	Cloud → all	signed, readable
`/agri/v1/ack/<node>`	node → Cloud	plain progress
`/agri/v1/report/<node>`	node → Cloud	sealed
`/agri/v1/status/<node>`	node → Cloud	plain, retained

E. Refusals are counted, not dropped

A report that fails to open is not discarded silently. It is counted, its reason is kept, and the dashboard displays it under REJECTED. A pipeline that hides its rejections is a pipeline that will one day be receiving nothing and look perfectly healthy.

In the reference campaign the panel read *"nothing refused — every report verified"* across all **48** reports, which is the statement worth making: not that the cryptography was exercised, but that it was exercised and nothing failed.

XII. The Transport Contract

A. Why MQTT and not HTTP

The Cloud must be able to reach the robot, not only the other way round. With HTTP the robot would have to poll for work; with MQTT it subscribes once and the broker pushes. The broker is also the only endpoint either side configures: neither has to know the other's address, and either can restart without the other noticing. That is the property that makes "the robot is a node on a network" an architectural fact rather than a turn of phrase.

It is additionally the protocol the rest of the agricultural-IoT world speaks, so the same robot could report to a different back end without a line changing in `agri/protocol.py`.

B. Topics, namespaced by node

Requests stay on a *flat* topic because a request addresses the fleet, not a specific node. Everything a node says is namespaced by its own identifier, and the Cloud subscribes with + to hear all of them, keeping nodes in a dictionary rather than a single variable. There is one mobile node today; the layout is what stops adding a second from being a protocol change.

C. Node kinds

Every status message carries a `node_kind` field: `"mobile"` for a robot that drives between stations, `"fixed"` for a stationary microcontroller bolted to a post. The field is carried now, with one mobile node and no fixed ones, so that adding fixed sensors later does not change the protocol — and so that the priority rule it exists for (prefer the mobile reading when a robot is parked at a station a fixed node also covers) has somewhere to live.

D. Three choices that were not defaults

a) QoS 1, not 0 or 2: Zero would drop a request while the robot is mid-reconnect. Two costs a four-way handshake for no benefit here, because the request identifier already makes a repeat harmless — and the store deduplicates on (label, timestamp, robot), which catches exactly the case QoS 1 permits and nothing else.

b) Status is retained; nothing else is: A dashboard opened at any moment should immediately know whether the robot is there, and MQTT's retained flag gives precisely that. Retaining *reports* instead would mean every reconnecting client re-receives the last measurement and files it again, which is how duplicates appear in a store that has no reason to have any.

c) A last will: If the robot process dies, the broker tells the Cloud on the node's own status topic. A dashboard that shows a robot as online because nobody said otherwise is worse than one that shows nothing.

E. Two failures the transport layer had to survive

Both were observed, and both are now pinned by the suite.

a) A broker that is not up yet: The robot node originally exited when it could not connect, which made the launch order of two terminals load-bearing — start the simulation before the broker and the robot was simply gone, with the failure scrolled off the top of a 140-line startup log. It now waits, retries with a bounded backoff, re-subscribes on *every* connect rather than only the first, says so when the broker goes away, and complains on the wall clock if none ever appears.

b) An exception inside an MQTT callback: `paho` does not guard its callbacks: an exception raised inside one kills its network thread. The observed symptom was a robot that looked perfectly healthy — its status timer kept publishing, because that runs on a different thread — while every request the Cloud sent was delivered to a client that was no longer reading. Forty minutes of a robot that would not move and a log that said nothing.

The cause was the status callback asking the driver for a pose before the simulator had published any odometry. Two rules came out of it and are asserted statically: no MQTT callback may let an exception escape, and `status()` must never raise even with no odometry at all — a status with no position is still worth sending, because "I am here and I am connected" is exactly what the Cloud needs at that instant.

XIII. The Cloud

A. One process, two faces

The Cloud is deliberately a single process presenting two interfaces: an MQTT client that talks to robots, and an HTTP server that talks to the operator. A third interface, a console on the main thread, is where orders are actually typed — the operator's seat is in the process that holds the private key.

Table XIV: The three moments a stored reading carries, each on the clock of the machine that owns it. End-to-end latency is derivable from the stored line alone.

field	the moment it records
`request_issued_at`	the Cloud signed the order
`measured_at`	the robot read the plant
`received_at`	the Cloud filed the report
`latency_s`	derived: issued → received

a) No web framework: The API has five endpoints and `http.server` is in the standard library. A framework would add a dependency, a version to pin and a deployment story, and would not make any of the five shorter. The one thing it would buy — production-grade concurrency — is not needed by a dashboard with one operator, and pretending otherwise would be the kind of choice that looks like engineering and is not.

B. Storage: JSON Lines, not a database

The brief asks for storage in a file of appropriate format. Appropriate here means openable by a human, appendable without rewriting, and diffable — which is JSON Lines: one visit per line, append-only, plus the images written out as real files, plus a CSV rebuilt on demand.

```
store/
  measurements.jsonl        append-only, one visit
    ↪ per line
  photos/P2,5R/2026....jpg  the photograph, as
    ↪ sent
  qr/P2,5R/2026....png      the QR code, as sent
  measurements.csv          rebuilt on export
  requests.csv              one row per order
```

A database would be defensible and is what a production system would use. It would also mean the evidence for every number in this report lives inside a binary file nobody can inspect during a demonstration. For a system whose entire claim is that the data arrived intact, being able to open the file and read the line is worth more than query speed over a few thousand rows.

The images come back out of the JSON because leaving them there would make the file unreadable — a 6 kB blob per line — and would mean the only way to look at a photograph is to write a program. Extracted, the line stays about 400 bytes and legible.

One corrupt line does not cost the campaign: the loader names the line it skipped and keeps the rest, because a truncated final line is what a kill during a write looks like and it is recoverable.

C. Three timestamps, because there are three moments

A reading has three distinct times, and the system originally kept one. The only honest answer to "how long did that sweep take" was then to watch a clock while it ran.

The column `timestamp` was removed rather than redefined, and the suite asserts it is gone — with three moments in play, a field called "timestamp" is a question, not an answer.

Orders are tracked in parallel in `requests.csv`, with `issued_at`, `completed_at` and `elapsed_s`. A mission that ends *without* filing every report still ends: the record closes on the terminal acknowledgement, so an abandoned sweep reports how long it ran before giving up rather than showing as running forever. The dashboard distinguishes the two verbs — "done" versus "gave up" — because the same numbers under the same word would report a failure as a success in the one place an operator looks for the answer.

D. The dashboard

The dashboard is a single page with no build step and no framework, polling `/api/state` every two seconds. It shows the greenhouse as a scale map with all 48 stations, the readings of the selected quantity, the plant photograph and QR image, live request progress, and the rejection panel.

Two problems in it are worth recording because both are geometric rather than aesthetic:

- **The two stations of an inner aisle collide when drawn.** At 0.10 m apart their marks touch at map scale. Each value chip is pushed away from its own row and joined to its mark by a stem, and the suite asserts the plain rendering really does collide and the corrected one does not — so the fix cannot be removed by someone who thinks it is decoration.
- **Text inherits the map's flip.** The SVG applies a y-negating transform so that $+y$ is north; every text element must counter-flip it or read mirrored. That counter-flip is written once, in a helper, and the suite asserts it is not open-coded per call site.

a) Access control: The dashboard API is protected by a random token printed at startup. `/media/` is behind the same token: photographs, QR images and the CSV export are measurements, so they sit behind the same gate as the API that describes them.

The token is offered once, in the URL, and the server's first act is to get rid of it: it sets an `HttpOnly; SameSite=Strict` cookie and redirects to the same page with the query string stripped. A credential that authorises driving the robot has no business living in the address bar, the browser history, the `Referer` of every outbound link and any proxy log in between. Repeated wrong tokens from one address earn a bounded lockout.

Authentication can still be disabled with `--dashboard-token ″`, but only together with `--http-host 127.0.0.1`; anything else refuses to start. It used to print a warning and serve anyway, and a warning at startup is read once and scrolls away. Sec. XIX covers this and the rest of the posture.

E. Stopping cleanly

The operator can stop the session from the console with Ctrl-C or from the browser with QUITTER. Both paths run *one* teardown function behind a lock, because both can happen at once and exporting the CSV twice from two threads truncates the file the first one is writing.

Table XV: Route cost over all 48 stations. Distances are computed along the real leg geometry, not estimated.

order	distance	band crossings
catalogue order, as issued	**312 m**	**45**
swept band by band	**40 m**	**3**
saving	**272 m (87%)**	42

The web path additionally has to exit a process whose main thread is blocked inside `input()`, which no exception raised on an HTTP worker thread can interrupt. It replies first, flushes the socket, and then terminates from a short-delayed thread — exiting from inside the handler resets the connection often enough to matter, and the browser then reports a failure for a shutdown that in fact worked.

The offline demo serves the same dashboard with the shutdown hook set to `None`, and the suite asserts that it refuses the request rather than killing its caller's interpreter.

XIV. Route Optimisation and the Energy Budget

A. The cost of the obvious order

The Cloud lists stations in catalogue order — `P1,1R`, `P1,1L`, `P1,2R`, `...` — which is right for a human reading a list and wrong for a robot driving a building. The two sides of one plant are in *different bands* (Table X), separated by an 8 m gutter, so `R` to `L` means driving to the end of the row, crossing at a headland, and coming back. Over 48 stations the catalogue order asks for that 45 times.

Measured against the same `route()` the driver actually flies, starting from the dock:

The signature of the defect was visible in the campaign log before it was diagnosed. Inter-station intervals within a row rose and then fell — $23, 28, 31, 34, 38, 39, 35, \ldots, 27, 24$ s — a bell curve, because a plant near a headland is cheap to cross at and a plant in the middle of the row is expensive from either end.

B. The order chosen, and why it is not a solver

Within a band the robot drives freely along x; leaving one costs a headland trip whatever the destination. The cheap route is therefore the obvious one — sweep a band end to end, cross once, sweep the next back — and the only remaining decisions are which end to start at and which direction to take. That is four combinations. All four are evaluated exactly, against the real route geometry, and the shortest wins.

A TSP solver would spend milliseconds rediscovering that the optimal tour of points on a line is to walk the line, and would then be free to return something else the day a constant changed.

a) The property that makes it safe to enable: **The order as issued is the fifth candidate, and it wins ties.** Adopting a longer route is therefore not a bug that could occur — it is something the function cannot do. The suite checks it over **720** random subsets of every size, and separately asserts that a request too small to benefit is returned untouched and that an unknown label does not raise.

Table XVI: Full 48-station campaign, before and after reordering. Times are wall-clock, from the Cloud's own request record.

	catalogue order	swept order
campaign duration	**1 505 s**	**601 s**
stations measured	**48/48**	**48/48**
reports rejected	**0**	**0**
floor camera resolved	**48/48**	**48/48**

b) The objection that was answered rather than accepted: The previous implementation carried a note saying reordering was omitted so that the acknowledgement stream would keep matching the request. That was a real concern and it is addressed directly: the mission retains the issued order alongside the driven one, every acknowledgement still names its own station, and the Cloud tracks progress by label. Nothing an operator reads was ever indexed by position.

C. Measured effect

The campaign runs in **40%** of the time for identical coverage and identical data quality.

D. Distance is measured; energy is modelled

This distinction is maintained in the code, in the CSV column names, and here, because collapsing it would put a fabricated number beside forty-eight real ones.

a) Measured: The distance driven is integrated from odometry in the driver, summed over successive positions rather than from the reported speed — the speed is sampled at 14 Hz and each sample would be held for the whole interval, accumulating a different error than the path itself has. A single step longer than 1.0 m is a respawn, not a drive, and is not counted. The figure inherits the odometry's drift, which is the honest behaviour: it measures what the robot *believes* it drove, which is the right basis for an energy estimate built on it.

b) Modelled: Gazebo models no battery and the platform carries no current sensor. Rather than display a percentage ticking down, `agri/survey.py` states four constants in the open, labelled in the source as ASSUMPTIONS, NOT MEASUREMENTS:

$$E = P_{\text{idle}} \cdot t_{\text{mission}} + P_{\text{drive}} \cdot t_{\text{driving}}, \qquad t_{\text{driving}} = \frac{d}{v_{\text{cruise}}} \tag{25}$$

with $P_{\text{idle}} = 18\,\text{W}$ (computer, lidar, two cameras, MQTT link), $P_{\text{drive}} = 45\,\text{W}$ (four wheel motors, while moving), $v_{\text{cruise}} = 0.45\,\text{m/s}$ and a nameplate pack of 120 Wh. The suite asserts v_{cruise} equals the driver's own `V_MAX`, so the estimate cannot drift away from the robot it describes, and asserts that t_{driving} can never exceed the mission duration — otherwise a bad odometry reading would invent drive energy out of nothing.

Table XVII: Energy budget. Distance and duration are measured; the three energy columns are derived from Eq. 25.

campaign	d	t	idle	drive
catalogue order	**312 m**	**1505 s**	*7.5 Wh* †	*8.7 Wh* †
swept order	**40 m**	**601 s**	*3.0 Wh* †	*1.1 Wh* †

† derived from the power model of Sec. XIV, whose constants are assumptions and not measurements.

E. The finding that matters more than the 87%

Applying Eq. 25 to the two campaigns:

Idle power is drawn whether or not the robot moves. On a 48-station sweep the robot spends far longer standing at marks — settling, focusing, photographing, encoding a QR code, sealing an envelope — than driving between them, so **shortening the route removes drive energy and leaves idle energy untouched: it attacks the smaller half of the bill.**

This is the more useful conclusion for a deployment. Having taken the easy 87%, further reduction is no longer a path-planning problem at all; it is a question of per-station dwell time, and therefore of the vision settling tolerance, the number of correction attempts, and the image encoding cost. Sec. XX returns to it.

F. What is exported

`requests.csv` keeps measured and modelled apart by name: `driven_m` and `planned_m` beside `energy_wh_est`, `idle_wh_est` and `drive_wh_est`. A request whose robot reported no distance leaves those cells **empty**, not zero — "this robot does not count metres" and "it drove nothing" are different facts, and the suite asserts the empty cells directly.

XV. The Pre-Flight Suite

A. The rule

Phase C continues the practice established in Phase B (Sec. V-H) and applies it more strictly: **every failure the project actually suffered becomes a check**. Not a class of failures, not a principle — the specific one, with the diagnosis written into the failure message so that the next person to see it red is told what it means.

The suite runs in about fifteen seconds and needs *no broker, no ROS 2 and no network*. That constraint is what makes it get run: a test that requires bringing up Gazebo is a test nobody runs before a small change.

B. Four kinds of check

The table above hides a distinction worth drawing out, because the last two categories are the ones that caught real defects.

a) Properties of geometry: Exhaustive rather than sampled: all 2 256 station-to-station routes cleared against the gutters; all 48 station clearances; all 48 QR codes encoding and decoding. These are cheap enough to run completely, so they are.

Table XVIII: The 750 pre-flight checks by area. One check runs only where `rclpy` is importable, so a machine with no ROS 2 reports 749. The suite grew by 366 checks when the security posture of Sec. XIX was addressed, and roughly a quarter of those are refusals.

area	checks
generated world and the plant meshes	63
generated assets: staleness and mesh paths	19
the plant camera: up axis and framing	21
texture budget, and the mesh rewriter	14
parking: one mark, named the same everywhere	20
the dashboard's mode selector	18
the ROS 2 package (read, never imported)	63
the two launch scripts	56
route order and the energy budget	44
the two modes: command and collector	43
key fingerprints, revocation, rotation	35
timing: issued, measured, received	34
broker TLS, credentials, no silent downgrade	33
the dashboard	30
catalogue geometry	25
free-space geometry and the brake	23
the dashboard's front door (token, cookie, lock-out)	22
end to end through a loopback broker	22
the store's hash chain	21
replay: freshness window and seen-set	18
station labels and their grammar	17
what the robot transmits about itself	17
Cloud on a second machine (key custody)	14
the floor camera and the painted mark	13
multi-node protocol	13
the dashboard's session controls	12
ECC: confidentiality and origin	11
sensor field structure	9
the offline demo, as a subprocess	7
QR round trip	5
paho client construction	4
repository hygiene	3
numpy ABI against rclpy	1
total	**750**

b) Refusals: Roughly a quarter of the suite asserts that something is *rejected*: a forged signature, a rewritten target, a foreign QR payload, a request for row 4, a label with side `M`, a missing key file, a second world generation over an already-painted scene. A system that accepts everything also passes every round-trip test.

c) Behaviour exercised, not string-matched: Where a static read would be weaker, the suite runs the real thing. The dashboard's session controls are exercised over real HTTP sockets against a real handler. The argument-parsing loop of `run_sim.sh` is *lifted verbatim out of the file* and executed under `bash`, because a regular expression asserting the loop looks correct is not the same as the loop being correct. The log filter is fed real launch output and its stdout inspected.

d) Structural assertions about the source: The remainder read the source and assert invariants that no runtime test would reach: that no MQTT callback can let an exception escape; that every topic the driver uses appears in the bridge configuration *and* that the bridge carries nothing nobody reads; that the launch file's computed resource path is actually applied rather

than computed and dropped; that `CRUISE_MPS` in the energy model equals the driver's `V_MAX`; that the private-key files are gitignored and none is tracked.

C. Three defects this suite caught after the fact

Each of these was working code that looked finished.

- **A variable computed and never used.** The launch file assembled the Gazebo resource path correctly and then never set it, so the robot's meshes could not resolve and it rendered as *nothing at all* — while every other part of the launch reported success. A value that is computed and dropped is the one kind of bug that reads as completed work. There is now a check that no other value in that file shares the fate.
- **An ordering assertion matching a comment.** Two checks confirmed the broker starts before the launch by comparing the positions of two strings. Adding an explanatory comment containing the words "ros2 launch" higher in the file broke both — correctly, but for the wrong reason. They are now anchored on the command, and the trap is documented where the next author will meet it.
- **A hygiene check that tested the wrong thing.** The private-key check scanned the working directory for `*.pem` and went red as soon as a developer generated keys, which is the normal case. It now asks git what is *tracked*, which is the actual property of interest.

D. The live diagnostic

One question cannot be answered offline — "why is RViz empty?" — so it has its own script, `tests/check_live.sh`, which runs against the *running* system and reports, in order, the four things that must be true before anything is drawn.

Its first act is to **refuse to answer when the launch is not running**. That behaviour was paid for: the diagnosis was twice run a minute after Ctrl-C, where `tf2_echo` reports "frame map does not exist" — a statement that is true, useless, and indistinguishable from the real fault. It also checks the odometry *before* the transform, because the transform is the symptom and the odometry is the cause, and reporting them in the other order sends the reader to the wrong file.

XVI. SWOT Analysis of Every Method

Every method in this system was chosen over alternatives, and every choice bought something at a price. This section states both. The quadrants are used in their strict sense: *strengths* and *weaknesses* are properties of the method as implemented here and are therefore verifiable in the code and the logs; *opportunities* and *threats* are external and forward-looking, and the threats in particular are what the transfer to real hardware will test.

A weakness listed below is not an admission of carelessness. It is the part of the design that would have to change first if the corresponding assumption failed, and knowing which part that is has more engineering value than a clean-looking table.

Table XIX: SWOT — Log-odds occupancy grid

Strengths Fusion is an addition, not a product and a renormalisation, so a scan is a sweep of += over an array: fast and numerically untroubled. Free space is represented explicitly, which is what makes frontier exploration possible at all. Clamping bounds the confidence any cell can accumulate, so a persistent spurious return cannot become unarguable.	**Weaknesses** The independence assumption between cells is false: a wall is a correlated structure, and treating its cells as independent lets a thin obstacle be eroded by beams that pass near it. Resolution is fixed at 0.10 m, which is coarse relative to a 0.80 m aisle. A moving obstacle leaves a smear that decays only as fast as the free-space evidence accumulates.
Opportunities The same grid accepts a second sensor without structural change: ultrasonic returns, which reflect off glass where lidar does not, fuse as additional log-odds increments. Multi-resolution variants exist if the aisle geometry ever demands them.	**Threats** Glass. A 2 D lidar at glancing incidence on a pane returns nothing, so the wall is not merely mis-mapped, it is absent, and the grid will confidently report free space where there is a barrier. This is the single most dangerous sim-to-real gap in the system, and no mapping parameter addresses it.

Table XX: SWOT — A* with an octile heuristic on an inflated grid

Strengths Optimal for the given grid and cost function, and complete: if it returns no path, no path exists at that inflation. Inflating obstacles by the footprint radius reduces the robot to a point, which is what makes an 0.80 m aisle tractable for a grid planner at all. The octile heuristic is admissible on an eight-connected grid, so optimality survives.	**Weaknesses** It plans for a point, so the resulting path takes no account of the robot's orientation — acceptable only because the base is holonomic. Inflation is isotropic, which over-inflates along the aisle where the base is narrow and under-inflates across it. Replanning is from scratch; there is no incremental repair.
Opportunities D* Lite would give incremental replanning at a fraction of the cost when the map changes locally. A costmap gradient rather than a binary inflation would let the path prefer the aisle centre instead of merely staying legal.	**Threats** Inflation radius and aisle width are in direct tension here: the margin that guarantees clearance can close the only route. A greenhouse whose aisles are narrower than this one — and 0.80 m is already tight — would make the planner report failure rather than risk, which is safe but useless.

Table XXI: SWOT — Pure pursuit path following

Strengths One parameter of real consequence, the lookahead, with a physical meaning that can be reasoned about rather than tuned blindly. Naturally smooth: it cannot produce the discontinuous commands a waypoint-snapping follower does. Cheap enough to run at 20 Hz beside everything else.	**Weaknesses** Corner cutting is intrinsic — the lookahead point is the target, not the path — and the amount cut scales with the lookahead, so the parameter trades tracking accuracy against stability with no setting that gives both. No feedforward: it reacts to the path, it does not anticipate it. It has no model of the actuators, so it will happily command a twist the base cannot produce.
Opportunities An adaptive lookahead scaled by speed and by local curvature is a small change with a measurable effect. The controller is also the natural point of comparison for the model-predictive alternative discussed in Sec. XX: identical path, identical disturbance, two control laws.	**Threats** Wheel slip on a wet greenhouse floor breaks the assumption that the commanded twist is the achieved twist, and pure pursuit has no mechanism to notice. In an aisle with 0.21 m of margin per side, uncorrected lateral drift reaches the gutter well before the next replan.

XVII. Experimental Results

[Section in preparation.]

Table XXII: SWOT — Colour-threshold fruit detection

Strengths Runs at frame rate on a CPU with no model, no training set and no dependency to install — which is exactly why it exists: it made the whole mission testable months before a learned detector could be. The excess-red form used here is invariant to illumination *scale*, so it survives shade.	**Weaknesses** **69%** recall and **66 of 111** map estimates matching no real berry. It cannot be tuned out of this: the test is invariant to illumination scale but not to illumination *colour*, and late-afternoon light through glass lowers G/R for the entire scene until every surface passes. It has no notion of shape, texture or context, so a red pipe is fruit.
Opportunities It remains a legitimate baseline, and a strong one for the report: a learned detector that does not beat it *on the same images* has proved nothing. It also survives as a cheap pre-filter to bound the region a heavier model must examine.	**Threats** Real greenhouse illumination varies far more than the simulated world's, and in the direction the threshold is weakest. Every hour of the day is a distribution shift, and the failure is silent — the detector reports confident fruit, not an error.

Table XXIII: SWOT — Corroborated fruit map (two passes and a baseline)

Strengths It gives the mission the one thing no detector improvement can supply: the ability to *disbelieve* a single observation. The two conditions reject different things — the pass count rejects the transient, the viewpoint baseline rejects the view-dependent artefact such as a specular highlight, which a detector at 15 Hz would otherwise corroborate thirty times from effectively one place. It is deliberately ignorant of detector confidence, so it keeps working unchanged when the threshold is replaced by a model.	**Weaknesses** It resolves clusters, not berries, and this is a hard limit rather than a tuning choice: measured fruit position error is 0.21 m while berries on one plant sit 0.05 m to 0.20 m apart, so the error exceeds the spacing. It also costs a scouting pass — corroboration is not free, it is paid for in survey time. A berry visible from only one side of the aisle can never accumulate a baseline and is therefore never confirmed.
Opportunities The same structure accepts a ripeness estimate per cluster, turning the harvest route into a scheduling problem over time rather than a tour over positions. Cluster persistence across sessions would let the robot return to fruit that was not ready yesterday.	**Threats** The rule assumes the fruit does not move between passes. Wind through open vents, or a picker working the same row, breaks that assumption in the direction that costs recall rather than precision — which is the safer direction, but still a loss.

Table XXIV: SWOT — Command-level protective stop with a single owner

Strengths Enforcement is structural rather than procedural: one node writes `/cmd_vel` and no publisher can reach the wheels without passing through it, so the guarantee does not depend on every future contributor remembering. Clearance is exact geometry validated against brute force (Sec. V-C), and the same function is used by the guard, by the escape manoeuvre and by the commissioning tool that certifies the distance.	**Weaknesses** It is a single point of failure by construction — that is the design — so a defect in it is a defect in all containment. It also brakes on a geometric criterion alone, with no notion of what the obstacle is: a leaf and a person produce the same stop. Cancelling translation while preserving rotation is correct for escape but means the robot can still turn in place against something it is already touching.
Opportunities The structure accepts a second, independent channel — bumper strips or ultrasonic rangefinders — without redesign, which is the normal route to a performance level a certification body will accept.	**Threats** A software-only protective stop on a kinematically driven base does not transfer as-is to a machine with real inertia: the stopping distance becomes a dynamic quantity, and ISO 13855 requires it to be measured, not assumed. Commissioning stage 0 exists precisely to measure it, and every clearance figure in this report is provisional until it does.

Table XXV: SWOT — Discrete state feedback for visual alignment

Strengths It removes a steady-state error that no proportional gain could: replayed against the logged stalls it reaches exactly zero offset where the previous law left 0.16 to 0.29 . The design is analytic — settling time and damping in, gains out — so the behaviour is specified rather than discovered, and the closed-loop poles are asserted in the regression suite.	**Weaknesses** The model is deliberately minimal, and its gain depends on range, so the poles are only verified stable across the 0.3 to 2.5 m band that matters here. It assumes the measured offset is the true offset, which the detector's false positives violate. It is single-input, single-output, so it cannot coordinate the lateral and heading corrections it is implicitly splitting.
Opportunities The multivariable extension — four wheel commands reduced to three body degrees of freedom, with the decoupling done in the gain matrix — is a direct continuation and the natural next contribution.	**Threats** No control law fixes an actuator limit. At 2 m an offset of 0.68 requires 1.36 m of lateral travel, which is 14 s at the 0.10 m/s ceiling against a 12 s timeout: the alignment was never going to succeed at any gain. The answer there is to refuse the station, not to tune the controller, and mistaking one for the other cost several runs.

XVIII. Acquired Capabilities and the Extension Path

A. What this section is

Phase C delivers a connected measurement node. Doing so exercises a strict subset of what was built and measured in Phases A and B: the platform, the twin, the safety layer, the lidar, the cameras, the regression discipline. Several substantial subsystems — mapping, SLAM, graph search, fruit detection, the five-DOF arm — are *present in the repository, validated, and not invoked by the delivered mission*.

That is a design outcome, not an omission, and it deserves stating precisely because the two readings are easy to confuse:

- These subsystems were **not** planned and left unbuilt. They were built, instrumented and measured, and their results — including two negative ones — are reported in Sections IV-M and XVII.
- They are **not** dormant code of unknown quality. Each is covered by the Phase B suite, and the Phase C suite asserts that Phase C never writes into the Phase B tree.
- They are **not** required by the delivered mission, because the mission operates in a catalogued space (Sec. VII).

They constitute an *extension path*: the capabilities the platform already has, which a wider deployment would need, together with what each would cost to bring back in.

B. The capabilities, and what each buys

Table XXVII is also the answer to the question a reader may reasonably ask on finishing Sec. VIII: the platform did not lose the ability to navigate an unknown space. It retains it, and the delivered mission declines to use it because the space is known.

Table XXVI: SWOT — Simulation-first development with a digital twin

Strengths	**Weaknesses**
It permits failures that would be unacceptable on hardware, and it supplies ground truth — the world knows where all **127** berries are — which makes every claim in Sec. XVII a measurement rather than an impression. Iteration is limited by a twenty-minute run, not by a greenhouse's availability or a season.	The twin is a model of what was already understood, so it cannot surface a phenomenon nobody thought to include, and the sensor models are optimistic in exactly the places that matter: the lidar sees glass, the camera has no motion blur, the floor does not slip. Success in simulation is evidence about the software, not about the robot.
Opportunities	**Threats**
The gap is narrowable in identified directions: an actuator model with transport delay and rate limits, domain-randomised illumination, and odometry drift injected from a calibrated model rather than assumed. The first of these is implemented and awaiting validation.	The characteristic failure of this method is confidence: a system that has only ever succeeded in simulation looks finished. The commissioning stages exist to make that confidence expensive to hold, by requiring a measured result on hardware before each capability is declared.

C. The extension that the measured results argue against

One row of Table XXVII deserves emphasis, because it is the one place where "we already built it" would be the wrong conclusion to draw.

The homemade SLAM front-end was implemented, instrumented, and found to be *worse* than the dead-reckoning prior it corrects (Sec. V-D). Phase C then solved the same underlying problem — knowing precisely where the robot is at the moment it takes a reading — by a different route entirely: it does not improve the global pose estimate at all. It accepts that odometry drifts, and closes the last centimetres visually against a painted physical feature whose position is known exactly (Sec. IX).

That comparison is worth more than either result alone. A global localisation solution accurate to **0.20 m** does not support a task whose labels are 0.10 m apart, and no incremental improvement to scan matching was going to change that by the required order of magnitude. A local, absolute, fiducial-based correction does support it — and it is cheaper, simpler, and verifiable from a single image. Where a task can be reposed so that global accuracy stops mattering, that is usually the better engineering.

D. Priority, if the work continues

Ordered by value against cost for this deployment, and drawing on the finding of Sec. XIV that per-station dwell time now dominates the energy budget:

1) **Replace the synthesised field with real probes.** This is the largest gap between the delivered system and a deployable one, and it is one function (Sec. X). Everything downstream — encoding, sealing, transport, storage, display — is already exercised end to end.
2) **Measure P_{idle} and P_{drive} on the bench.** Two numbers turn Eq. 25 from a stated model into a measured budget, and the autonomy figure a fleet operator actually needs follows immediately.
3) **Attack per-station dwell time**, which is where the remaining energy is: the visual settling tolerance, the number of correction attempts, and image encoding cost.
4) **Re-enable obstacle handling** (A* over a local costmap seeded from the catalogue) before any deployment sharing aisles with people. The brake already stops for an unexpected object; it has nowhere to route around one.
5) **Fixed ESP nodes.** The protocol already carries `node_kind` and namespaces every topic per node (Sec. XII); the Cloud already keeps nodes in a dictionary. This is the cheapest capability increase available, because the interface for it was built and tested before there was anything to plug into it.

XIX. Security Posture

A. Why this section exists

Section XI describes a sealed, signed report and a signed request. Read on its own, that reads like a solved problem, and it is not one. Encryption answers a narrow question — can a third party read this message, and did the named sender write it — and a deployed system fails in ways that leave both answers "yes".

An order captured off the broker and published again a week later carries the Cloud's own signature. A stranger answering *hold* to every handover fills the robot's outbox and produces a campaign with no data and no errors. A filed reading edited on disk was verified on arrival and is verified by nothing afterwards. None of those are defeats of the cryptography; all three were open in the version of this system described by the previous section.

This section states the threat model, lists what the delivered code now enforces, and — at least as importantly — says what it does not, so that the boundary is a decision on record rather than something a reader has to infer.

B. Threat model

The system is assumed to run on a small network the operator does not exclusively control: a farm LAN with a shared wireless segment, other machines on it, and physical access to the greenhouse by people who are not the operator.

Explicitly *out* of scope: an attacker who can break P-256 or AES-256-GCM, and one with root on the Cloud while it is running. Neither is defensible at this scale, and pretending otherwise would make everything else here less believable.

C. What the delivered code enforces

a) A signed order stops being an order: `agri/replay.py`. A signature says who wrote a message, not that this is the first time it has been read. Requests now carry a freshness window (120 s) *and* a seen-set of request identifiers. Neither works alone: freshness leaves the window open, and a seen-set alone would have to remember every identifier forever and across reboots — which is exactly when a replay would be attempted. Together they need no persistence and leave no gap, because an identifier older than the window is refused on age before the set is consulted.

Table XXVII: Capabilities built and measured in Phases A–B, their status against the Phase C mission, and the condition under which each stops being optional. "Measured" refers to the results in Sec. XVII.

Capability	Status after Phases A–B	Why Phase C does not use it	What would require it
Occupancy mapping (log-odds, 2D lidar)	**Works.** Interior reconstructed to -4 **cm** / $+10$ **cm**, **100%** floor coverage, **0.0%** phantom clutter.	The greenhouse is generated from a catalogue that predates the robot; a map would be a lossier copy of a file already on disk.	A greenhouse whose layout is *not* surveyed, or one where gutters are moved between seasons without the catalogue being updated.
A* path planning	**Works**, and was tuned through inflation and clearance problems in a 0.16 m gap.	Free space is four corridors joined at both ends; the closed-form route cannot return a novel answer, which in that clearance is a feature (Sec. VIII).	Mobile obstacles — a trolley, a person, a second robot — or any layout not decomposable into parallel bands.
Correlative scan-matching SLAM	**Measured worse** than the calibrated odometry prior it was meant to correct; disabled in the production configuration (Sec. V-D).	Not used, and would not be used even if it worked: the visual mark alignment closes position against a physical feature of the building, which is stronger than either.	Nothing, as implemented. A loop-closing back-end would be a new subsystem, not a re-enabling of this one.
Colour fruit detection and map placement	**Partially works.** **77%** recall, **0.22 m** localisation error, roughly two of three map estimates spurious.	Phase C photographs the plant for a human and does not classify it. The detector's error is a quarter of the plant spacing — enough to identify the plant, not the berry.	Any autonomous decision taken *from* the image: ripeness triage, disease flagging, yield estimation.
Five-DOF arm and gripper	**Works.** Pick-and-place sequence validated in both simulators.	The mission measures and photographs; it does not touch the plant.	Sampling that requires contact — leaf collection, substrate probing, selective harvesting.
Behaviour-tree mission supervision	**Works.** Multi-trip harvesting with dynamic re-planning.	A Phase C mission is a list of stations with a defined end. A queue and a loop express that exactly; a behaviour tree would express it loosely.	Missions with conditional branches: recharge when low, re-measure on an anomaly, defer a blocked station and return to it.

Table XXVIII: Who is assumed capable of what. The middle row is the one that drives most of the design: an attacker who never breaks a key can still do a great deal.

actor	assumed capability
network observer	reads every packet on the segment; learns topics, timings and message sizes
network participant	connects to the broker and publishes on any topic; captures and replays messages verbatim
disk access	reads and writes files on the Cloud machine, including the store and the key directory
key holder	has one long-term private key, e.g. from a robot that was stolen or decommissioned

`request_id` serves as the nonce; it is already random, already unique and already inside the signature.

The failure mode this introduces is a clock, not an attacker, and the code treats it that way: a future-dated order is reported as clock skew and never as an attack, an accepted order already consuming half the window logs a warning, and every refusal prints the disagreement in seconds alongside `timedatectl`.

b) The handshake is signed in both directions: The negotiation messages were left plain, on the reasoning that a forged *send* only makes the robot offer a reading it was about to offer anyway. That reasoning missed the other grant. *Hold* means keep it, and the robot obeys by putting the reading in a bounded outbox. Answered to every offer, it fills 48 slots, starts dropping the oldest readings, and leaves a robot reporting itself perfectly healthy while the campaign produces nothing — for the cost of one MQTT publish. Queries are now signed by the Cloud and replies by the node, each with a key the other already holds.

c) The transport can be closed: `deploy/`. ECIES hides a reading; it does not hide that `agri/v1/report/youbot-01` published 6 kB at 14:02, and for a commercial greenhouse that traffic pattern is a map of the operation. Both programs accept a CA, a client certificate and a key; naming a CA turns TLS on and moves the port from 1883 to 8883 without a second decision. Neither side ever falls back to plaintext — a misconfiguration is fatal, because a system that silently downgrades is worse than one with no TLS, since somebody will believe it. The reference `mosquitto.conf` does not listen on 1883 at all, and the ACL's `%u` patterns stop one node publishing as another and stop any node issuing orders.

There is deliberately no password flag: `/proc/<pid>/cmdline` is world-readable, so a password in `argv` is one that `ps` prints to anyone with a shell.

d) The dashboard credential leaves the URL: A token in a query string lives in the address bar, the browser history, the `Referer` of every outbound link, and any proxy log in between — for a credential that authorises driving the robot and shutting the simulation down. It now moves into an `HttpOnly; SameSite=Strict` cookie on first contact and the page redirects to a clean address. Repeated wrong tokens earn a bounded lockout. Disabling authentication is still possible and is now permitted only when the server is bound to loopback: a property the program checks, rather than a warning that scrolls past.

e) A filed reading that changes says so: `agri/cloud/store.py`. Every report is verified on

arrival and was then written in plain text, after which the cryptography had nothing further to say about it. Each line now carries the SHA-256 of its own content chained to the hash before it, so editing one reading breaks that line, repairing that line breaks the next, and deleting a line is caught by the chain even though every remaining line still hashes correctly.

f) A key can be named and retired: `agri/trust.py`. Keys get a 64-bit fingerprint in four readable groups, printed at startup by both programs, so "do we hold the same key" is settled in ten seconds rather than by an afternoon of signature errors. A revocation list is consulted at startup and both programs *refuse to run* on a listed key. `-rotate` archives the old pair before touching anything — reports already sealed to a private key can be opened with nothing else — and revokes the old public key before writing the new one.

D. What it does not do, and what that would cost

E. The rule the protocol now follows

One line makes the message table readable without consulting the code: *everything that can change what the other side does is signed; everything that only reports is not*. A request, a query and a reply all decide something and all carry signatures. An acknowledgement and a status are observations, and do not.

That rule is worth stating because the alternative — signing everything — is the choice that looks safer and teaches nothing. It would leave a reader unable to tell which messages actually matter, and it is the same reflex that would have encrypted a request whose contents are "measure `P2,5R`".

F. What was found by writing this down

Two of the gaps above were closed only because the exercise of enumerating them made them impossible to keep deferring, and one defect was found by a linter rather than a test: a dictionary literal in `agri/envelope.py` named two different values `plant`, which Python resolves last-wins in silence. Every report ever filed carried a coordinate pair where the plant index belonged, beside a row index that was a plain integer. Nothing downstream read the field, so nothing ever broke — it was simply wrong in the file, which is the class of defect only found by reading the file or by a tool that reads it for you.

XX. Discussion and Limitations

[Section in preparation.]

XXI. Conclusion and Future Work

[Section in preparation.]

References

[1] C. W. Bac, E. J. van Henten, J. Hemming, and Y. Edan, "Harvesting robots for high-value crops: State-of-the-art review and challenges ahead," *Journal of Field Robotics*, vol. 31, no. 6, pp. 888–911, 2014.

[2] E. J. van Henten, J. Hemming, B. A. J. van Tuijl, J. G. Kornet, J. Meuleman, J. Bontsema, and E. A. van Os, "An autonomous robot for harvesting cucumbers in greenhouses," *Autonomous Robots*, vol. 13, no. 3, pp. 241–258, 2002.

[3] J. Borenstein and L. Feng, "Measurement and correction of systematic odometry errors in mobile robots," *IEEE Transactions on Robotics and Automation*, vol. 12, no. 6, pp. 869–880, 1996.

[4] O. Michel, "Webots: Professional mobile robot simulation," *International Journal of Advanced Robotic Systems*, vol. 1, no. 1, pp. 39–42, 2004.

[5] T. Foote and M. Purvis, "REP-103: Standard units of measure and coordinate conventions," 2010, rOS Enhancement Proposal.

[6] B. E. Ilon, "Wheels for a course stable self-propelling vehicle movable in any desired direction on the ground or some other base," 1975, u.S. Patent 3 876 255.

[7] H. P. Moravec and A. Elfes, "High resolution maps from wide angle sonar," *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, vol. 2, pp. 116–121, 1985.

[8] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. Cambridge, MA: MIT Press, 2005.

[9] J. E. Bresenham, "Algorithm for computer control of a digital plotter," *IBM Systems Journal*, vol. 4, no. 1, pp. 25–30, 1965.

[10] T. Lozano-Pérez, "Spatial planning: A configuration space approach," *IEEE Transactions on Computers*, vol. C-32, no. 2, pp. 108–120, 1983.

[11] E. B. Olson, "Real-time correlative scan matching," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2009, pp. 4387–4393.

[12] B. Yamauchi, "A frontier-based approach for autonomous exploration," in *Proc. IEEE Int. Symp. Computational Intelligence in Robotics and Automation (CIRA)*, 1997, pp. 146–151.

[13] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.

[14] R. C. Coulter, "Implementation of the pure pursuit path tracking algorithm," Robotics Institute, Carnegie Mellon University, Tech. Rep. CMU-RI-TR-92-01, 1992.

[15] J. M. Snider, "Automatic steering methods for autonomous automobile path tracking," *Robotics Institute, Carnegie Mellon University, Tech. Rep. CMU-RI-TR-09-08*, 2009.

Appendix A
Reproducing Every Experiment

[Appendix in preparation.]

Appendix B
Complete Parameter Tables

[Appendix in preparation.]

Appendix C
Topic and Frame Contract

[Appendix in preparation.]

Appendix D
Failure Modes and Diagnosis

[Appendix in preparation.]

Table XXIX: Deliberate boundaries. Each row is a real weakness of the delivered system; the third column is what closing it would take, so that the boundary is a decision rather than an oversight.

gap	why it is open	what would close it
The revocation list is not distributed	Revoking a key on the Cloud tells the robot nothing. Each machine holds its own list and they are reconciled by `scp`.	X.509 certificates with a CRL or OCSP, i.e. a real PKI. At two nodes that is more machinery than the problem.
Key age is a reminder, not a validity period	A raw PEM carries no expiry — that is precisely what a certificate adds. The birthday sits in an unsigned sidecar the key holder could edit.	The same PKI. Until then the honest claim is that an old key is mentioned at every startup.
The hash chain lives on the same disk as the data	An attacker with write access can recompute the whole chain. What the chain buys is that a surgical edit is detectable and a forgery must rewrite the file to its end.	Pinning the tip somewhere out of reach: an off-box nightly copy, or a signature from a key that is not on the machine. `-verify` prints the tip so that this is possible.
The dashboard is plain HTTP	The cookie is `HttpOnly` and `SameSite`, but not `Secure` — a `Secure` cookie on an HTTP origin is silently discarded, which would look like authentication that does not stick.	A TLS reverse proxy, and then setting `Secure` and HSTS. No code change.
One shared token, no accounts	Every operator uses the same credential; there is no record of who issued which request. `requests.csv` attributes orders to the Cloud, not to a person.	Per-operator credentials and an actor field inside the signed request. The protocol has room; the deployment has no identity provider.
Dependencies are unpinned	`pip install -e .` resolves `paho`, `cryptography`, `opencv` and `Pillow` at install time, so two machines can run different code.	A lock file, and a hash-checking install. Worth doing before this is handed to anyone.
Acks and status are unsigned	They only report, so forging one lies to a dashboard rather than steering a robot — a deliberate line, stated in `agri/protocol.py`.	Signing them too. The cost is one verification per telemetry message at 0.2 Hz, which is nothing; the reason not to is that it would blur the rule that makes the protocol legible.

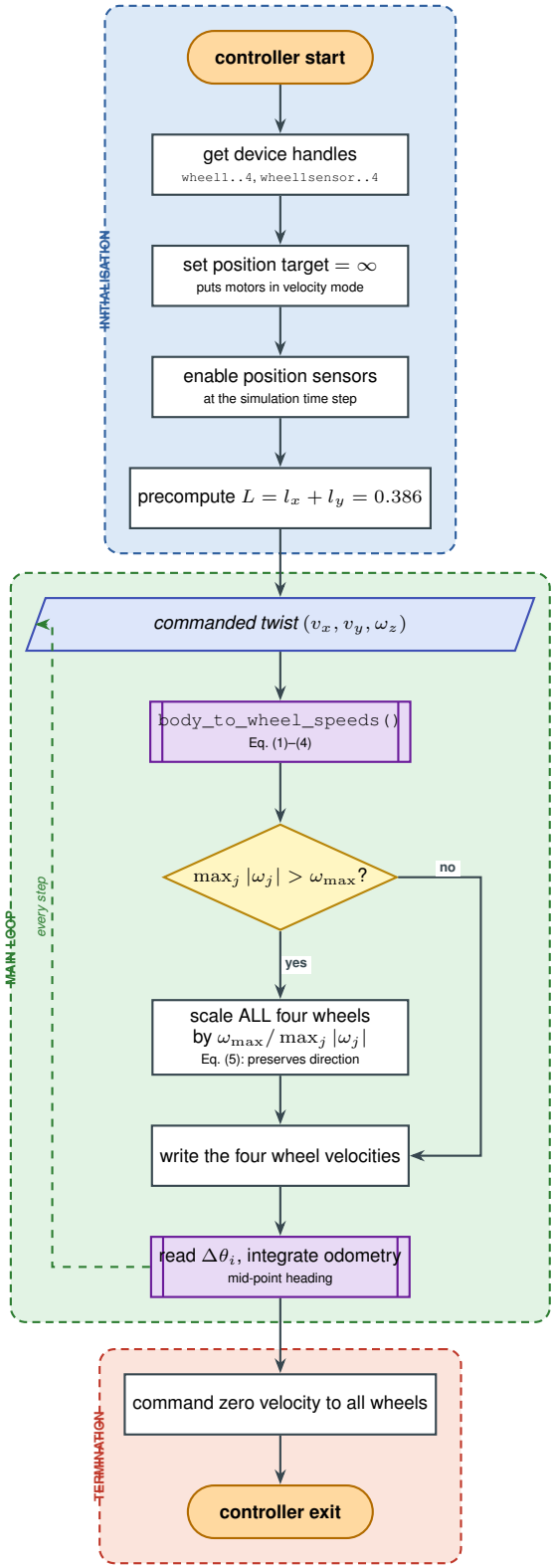
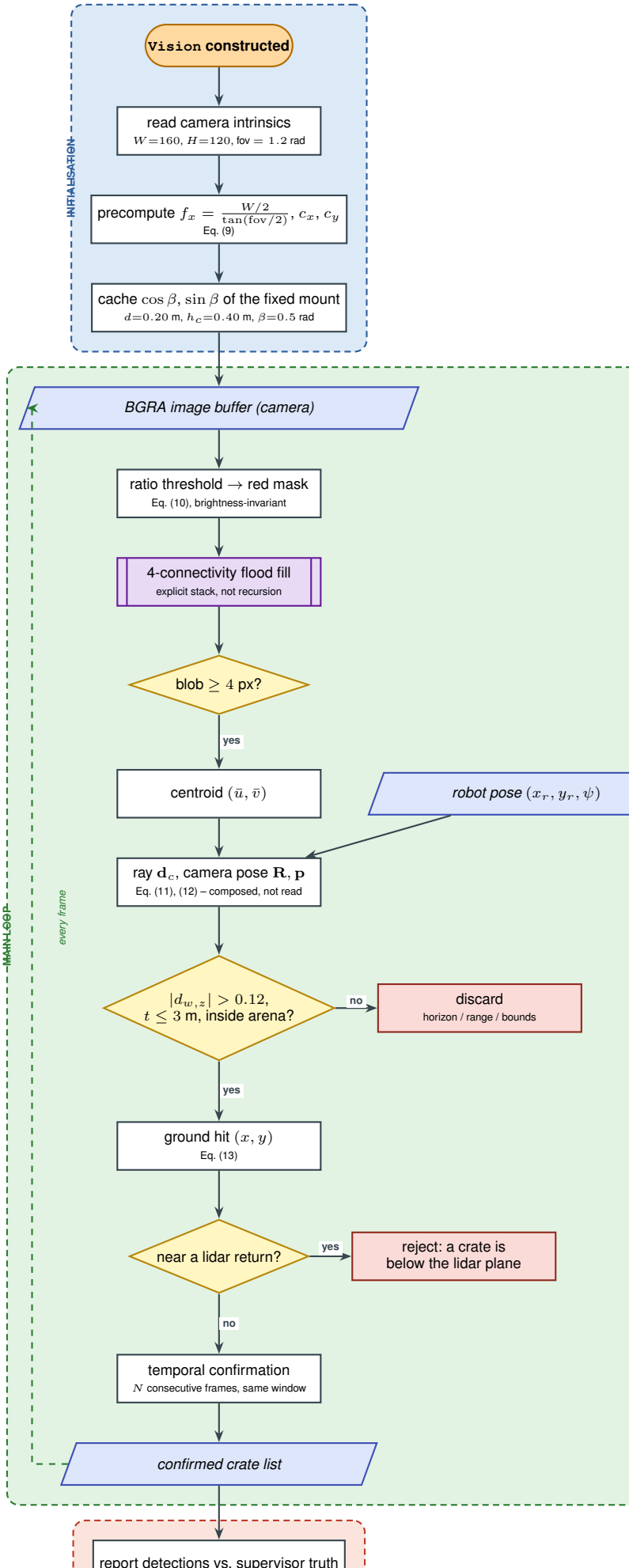
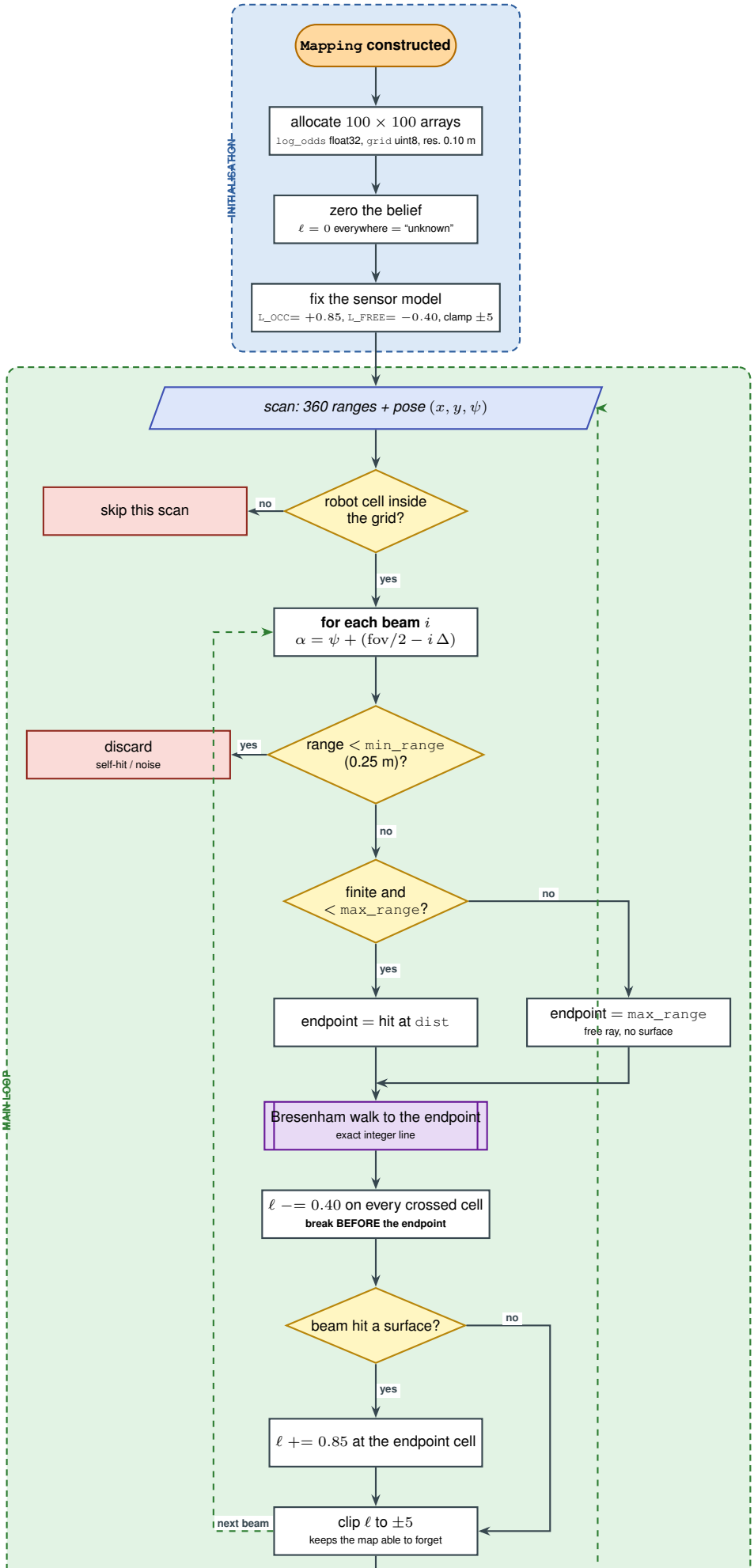
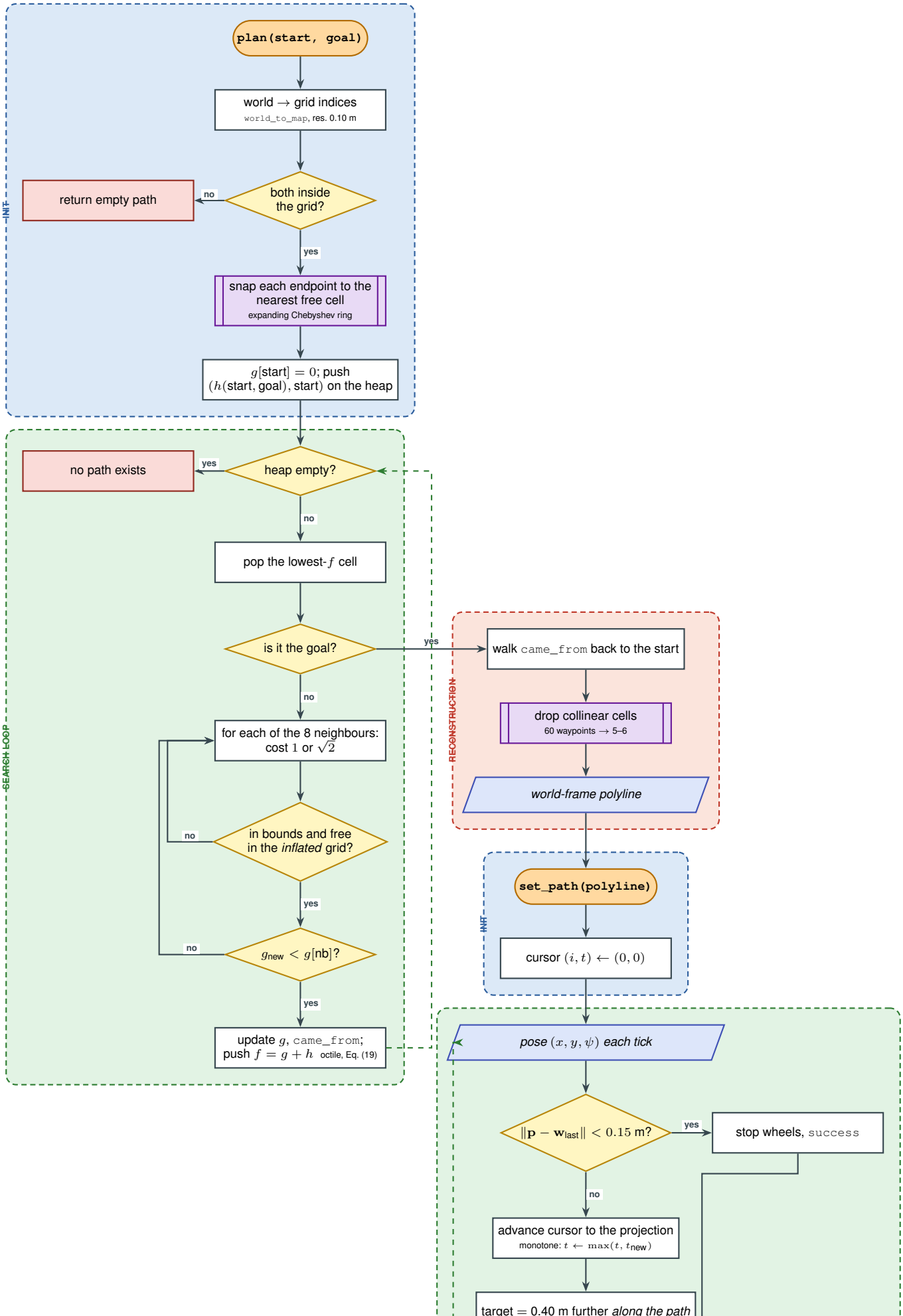


Figure 2: Mecanum base controller (Phase A). The initialisation band holds everything computable once, including the Webots idiom of setting a motor's position target to infinity to obtain velocity control. The saturation test is the only branch in the loop, and it scales all four wheels together so that a speed limit costs magnitude and never direction. The termination band exists because a controller that exits without zeroing the wheels leaves the base coasting into the next run.




INITIALISATION
Mapping constructed
allocate 100×100 arrays
log_odds float32, grid uint8, res. 0.10 m
zero the belief
$\ell = 0$ everywhere = "unknown"
fix the sensor model
L_OCC= $+0.85$, L_FREE= -0.40, clamp ± 5
MAIN LOOP
scan: 360 ranges + pose (x, y, ψ)
robot cell inside the grid?
no
skip this scan
yes
for each beam i
$\alpha = \psi + (\text{fov}/2 - i\,\Delta)$
range < min_range (0.25 m)?
yes
discard
self-hit / noise
no
finite and < max_range?
no
yes
endpoint = hit at dist
endpoint = max_range
free ray, no surface
Bresenham walk to the endpoint
exact integer line
ℓ −= 0.40 on every crossed cell
break BEFORE the endpoint
beam hit a surface?
no
yes
ℓ += 0.85 at the endpoint cell
clip ℓ to ± 5
keeps the map able to forget
next beam


plan(start, goal)
world → grid indices
world_to_map, res. 0.10 m
both inside the grid?
no
return empty path
yes
snap each endpoint to the nearest free cell
expanding Chebyshev ring
g[start] = 0; push (h(start, goal), start) on the heap
INIT
heap empty?
yes
no path exists
pop the lowest-f cell
is it the goal?
yes
no
for each of the 8 neighbours: cost 1 or √2
in bounds and free in the inflated grid?
no
yes
g_new < g[nb]?
no
yes
update g, came_from; push f = g + h octile, Eq. (19)
SEARCH LOOP
walk came_from back to the start
drop collinear cells
60 waypoints → 5–6
world-frame polyline
RECONSTRUCTION
set_path(polyline)
cursor (i, t) ← (0, 0)
INIT
pose (x, y, ψ) each tick
‖p − w_last‖ < 0.15 m?
yes
stop wheels, success
no
advance cursor to the projection
monotone: t ← max(t, t_new)
target = 0.40 m further along the path

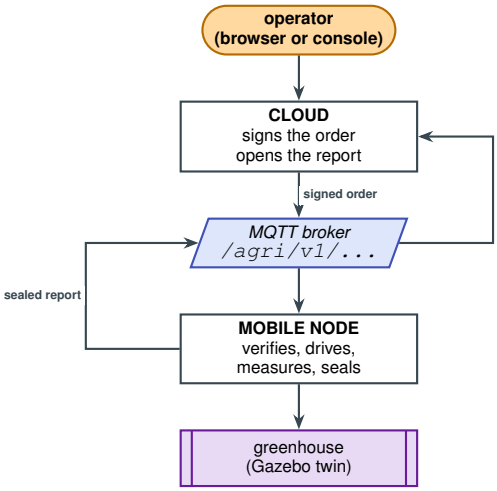


Figure 6: Phase C at a glance. The two processes never address each other: the broker is the only endpoint either configures. The order is *signed* and readable; the report is *sealed* and is not (Sec. XI).